TÉCNICO UNIVERSITARIO EN PROGRAMACIÓN

TRABAJO PRÁCTICO · PYQT6 + QT DESIGNER

Login y registro con Qt Designer

Vas a construir una aplicación de escritorio de verdad: desde una pantalla vacía hasta un login con registro que valida lo que se escribe, avisa con claridad y está ordenado en clases. La pantalla la vas a **dibujar**. El comportamiento lo vas a **programar**. Y la parte que de verdad se aprende acá es **conectar las dos cosas sin que se mezclen**.

> **Diseñamos visualmente, programamos el comportamiento y conectamos ambas partes con buenas prácticas.**

PYQT6 · QT DESIGNER · ARCHIVOS .UI · UIC.LOADUI() · SEÑALES · VALIDACIÓN · QT STYLE SHEETS · POO · GIT

MODALIDAD Individual o en parejas de hasta dos personas	ANTES DE EMPEZAR Guía de exploración de PyQt, clases 1 a 3	SE ENTREGA Un repositorio con `.ui`, código, README y capturas
DURACIÓN SUGERIDA Tres clases de taller + trabajo en casa	INTEGRANTES Nombre y apellido	REPOSITORIO https://github.com/...

00

Cómo se trabaja este TP

Leelo una vez: todas las etapas siguen el mismo ritmo

Cada etapa importante avanza en siete pasos. No te salteés los del final: son los que convierten «lo copié y anduvo» en «sé por qué anda».

EL PROBLEMA
Qué queremos resolver y por qué todavía no podemos.

LA IDEA
El concepto nuevo que hace falta. Pocas palabras.

LA DEMOSTRACIÓN
Un ejemplo chico que funciona. Escribilo y ejecutalo.

PROBÁ
Una modificación que hacés vos. A veces, romperlo a propósito.

PENSÁ
Preguntas para razonar. Se contestan con argumentos, no con código.

INVESTIGÁ
Algo que esta guía no te dice a propósito. Se busca en la documentación.

COMPROBÁ
Condiciones verificables. Si alguna no se cumple, no avances.

NIVEL 1 · OBLIGATORIO
Lo mínimo para aprobar. Todas las etapas lo marcan.

NIVEL 2 · INTEGRACIÓN
Lo que convierte dos pantallas sueltas en una aplicación. Esperado para una buena nota.

NIVEL 3 · OPCIONAL
Desafíos «¿y si...?». Con pistas, sin soluciones.

SOBRE LAS SOLUCIONES

Esta guía **no** te da el código completo de la aplicación. Te da ejemplos chicos que funcionan, esqueletos para completar y pistas plegadas (▸ **Pista**) para cuando te trabes. Abrí una pista **después** de intentar, no antes. Las respuestas que escribas en los recuadros quedan guardadas en este navegador, y salen en el PDF si lo imprimís.

Lo que ya traés puesto

DE LA GUÍA DE PYQT
`QApplication`, widgets, `clicked.connect()`, layouts y una ventana como clase que hereda de `QWidget`.

DE LAS CLASES 1 Y 2
`__init__`, `self`, herencia, `super()`, encapsulación y la diferencia entre **ES UN** y **TIENE UN**.

DE LA GUÍA DE MYSQL
Una clase que concentra una responsabilidad, el `.gitignore` y la idea de que **la interfaz no tiene por qué conocer los datos**.

Decisiones técnicas de este TP

DECISIÓN	POR QUÉ
PyQt6	Es la versión que usamos en la guía de exploración: todo lo que ya probaste sigue valiendo. Si encontrás ejemplos de PyQt5, usá el «traductor» de la sección 10 de esa guía.

DECISIÓN	POR QUÉ
Qt Designer desde `PySide6-Essentials`	Es la forma más simple que verificamos para tener Designer en Windows y en Mac — incluidas las Mac con chip Apple (M1 a M4)— con un solo `pip install`. Lo usamos **solo para dibujar**; el programa usa PyQt6.
Cargar el `.ui` con `uic.loadUi()`	El diseño queda en el `.ui`, que podés volver a abrir y cambiar en Designer cuando quieras. No lo convertimos con `pyuic6` en un `.py` generado: ese archivo es largo, nadie lo edita a mano y se pisa cada vez que regenerás.
Una clase por pantalla	Cada pantalla sabe responder a sus botones. Nadie más.
Autenticación simulada	Sin base de datos todavía. Pero lo que se puede hacer bien hoy, se hace bien: las contraseñas nunca se guardan como texto.

El recorrido

0 de 45 comprobaciones marcadas

01	Entender qué vamos a construir	qué hay y qué pasa
02	Preparar el entorno	venv · PyQt6 · Designer
03	Aprender Qt Designer explorando	.ui · objectName · layouts
04	De `.ui` a Python	uic.loadUi · rutas · errores
05	Aplicar POO	una clase por pantalla
06	Conectar eventos	señales · navegación
07	Validar los formularios	reglas · mensajes
08	Simular el funcionamiento	servicio · hash
09	Mejorar la experiencia visual	Qt Style Sheets
10	Organización del proyecto	módulos · responsabilidades
11	Buenas prácticas y seguridad	revisión de código
12	Desafíos progresivos	niveles 1, 2 y 3
13	Conexión con el futuro	hacia MySQL
14	Entrega y GitHub	README · commit · push
15	Autoevaluación y criterios	checklist · rúbrica

01 Entender qué vamos a construir

Antes de abrir cualquier programa: qué se ve, qué se toca y qué pasa

NIVEL 1

EL PROBLEMA

Una persona abre tu programa. Si ya tiene cuenta, entra. Si no, se registra. ¿Qué tiene que ver, qué puede tocar y qué pasa cada vez que toca algo?

DOS PANTALLAS, UNA APLICACIÓN

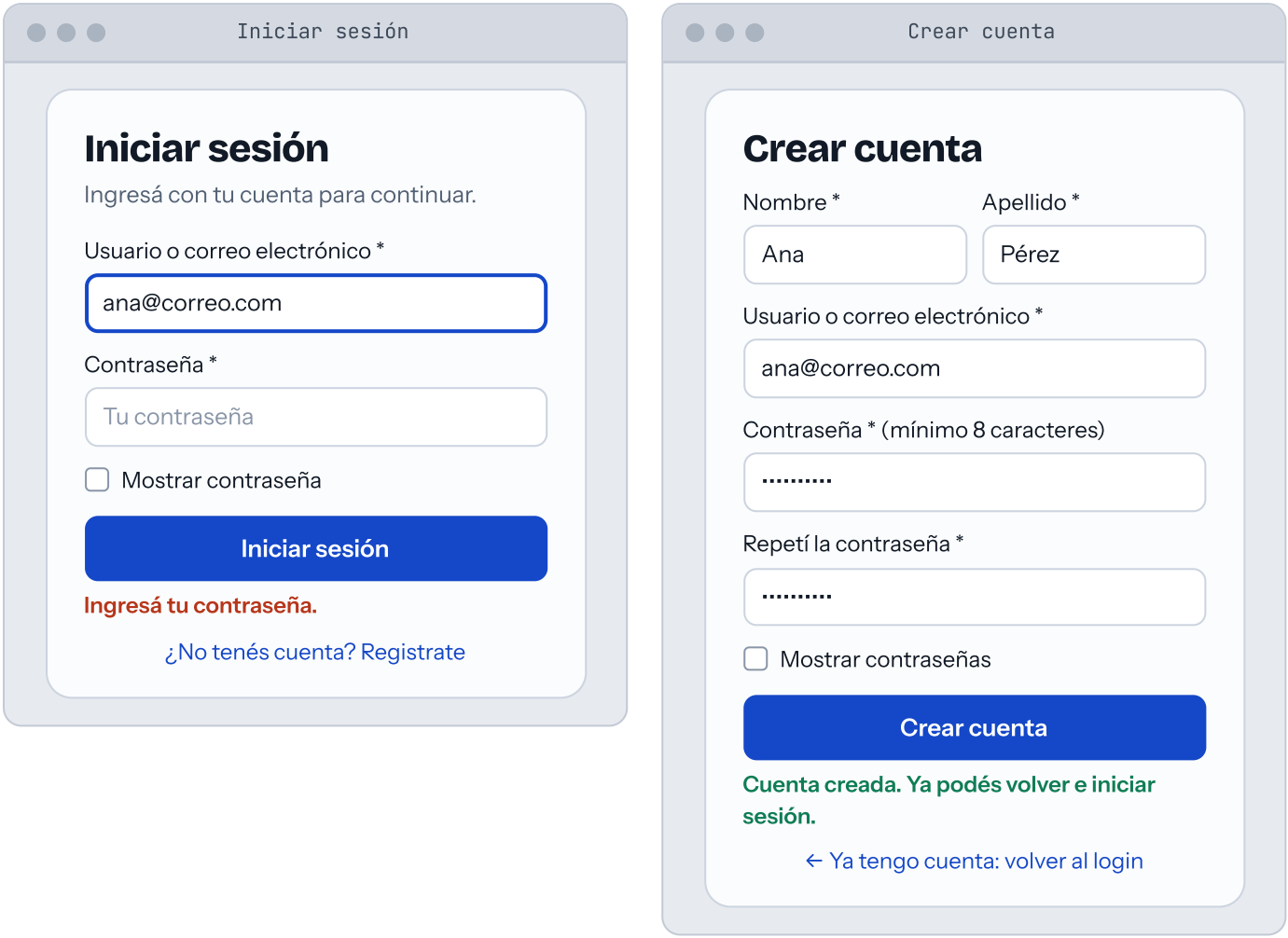


ILUSTRACIÓN DE REFERENCIA — dibujada para esta guía, **no es una captura de PyQt**. Muestra una posibilidad: tarjeta centrada, un botón principal que se distingue, mensajes con color y texto. Tu versión se va a ver distinta según tu sistema operativo y tus decisiones, y está bien que sea distinta.

LOGIN

- Usuario o correo electrónico.
- Contraseña, oculta mientras se escribe.
- Opción para mostrar u ocultar la contraseña.
- Botón para iniciar sesión.
- Acceso a la pantalla de registro.
- Un lugar para los mensajes de validación.

REGISTRO

- Nombre y apellido.
- Usuario o correo electrónico.
- Contraseña y confirmación de la contraseña.
- Botón para registrarse.
- Acceso para volver al login.
- Un lugar para los mensajes de validación.

Cómo se pasa de una a otra

Login

Iniciar sesión → revisa los datos → muestra un mensaje.

`«Registrate» → btn_ir_registro`

`btn_volver_login ← «Volver»`

Registro

Crear cuenta → revisa los datos → guarda (simulado) → muestra un mensaje.

PENSÁ · ANTES DE DISEÑAR NADA

1. ¿Qué información necesita cada pantalla para hacer su trabajo?

2. ¿Qué elementos son solamente visuales (se ven, pero no hacen nada)?

3. ¿Qué elementos requieren programación?

4. ¿Qué debería ocurrir cuando se presiona cada botón? Uno por uno.

5. ¿Qué responsabilidades pertenecen a la interfaz y cuáles a la lógica?

PROBÁ · CLASIFICÁ CADA COSA

Marcá dónde va a vivir cada una. Algunas tienen trampa: leelas despacio.

ELEMENTO	DISEÑO (`.UI`)	COMPORTAMIENTO (`.PY`)
El título «Iniciar sesión»	☐	☐
Que la contraseña se vea con puntos	☐	☐
Que al tildar «Mostrar» se vea la contraseña	☐	☐
El color del botón principal	☐	☐
Revisar que el usuario no esté vacío	☐	☐
Decidir *qué* mensaje mostrar	☐	☐

ELEMENTO	DISEÑO (`.UI`)	COMPORTAMIENTO (`.PY`)
El lugar donde aparece el mensaje	☐	☐
Pasar a la pantalla de registro	☐	☐
El orden en que el Tab recorre los campos	☐	☐

▼ ▸ PISTA (ABRILA DESPUÉS DE MARCAR)

Hacete una sola pregunta por fila: **¿esto describe cómo es la pantalla al abrirse, o decide qué pasa cuando alguien hace algo?** Lo primero va en el `.ui`. Lo segundo, en Python. La trampa está en las filas 2 y 3: el estado inicial (puntos) es diseño; el cambio al tildar es comportamiento. Lo mismo con el mensaje: el lugar es diseño, el texto que aparece lo decide el código.

LA IDEA

LA INTERFAZ DESCRIBE

- Qué widgets hay y dónde.
- Cómo se llaman (`objectName`).
- Cómo arrancan: textos, campos ocultos, orden del Tab.
- Cómo se ven.

LA LÓGICA DECIDE

- Qué pasa cuando se toca cada cosa.
- Qué datos son válidos.
- Qué mensaje mostrar.
- A qué pantalla ir. Quién puede entrar.

COMPROBÁ ANTES DE SEGUIR

- ☐ Puedo decir qué pasa al tocar cada control interactivo de las dos pantallas.
- ☐ Sé qué cosas van a vivir en el diseño y cuáles en Python, y puedo justificar los casos dudosos.

02

Preparar el entorno

Una carpeta, un entorno virtual, una librería y una herramienta

NIVEL 1

EL PROBLEMA

? **PyQt no viene con Python. Qt Designer tampoco. Y tener uno no te da el otro.**

LA IDEA

PYQT6 · UNA LIBRERÍA

Código que **tu programa importa**. Sin ella, `from PyQt6 import uic` falla. Quien ejecute tu aplicación la necesita: por eso va en `requirements.txt`.

QT DESIGNER · UNA HERRAMIENTA

Un **programa aparte**, con ventanas, para dibujar pantallas. Guarda archivos `.ui`. Tu aplicación no lo necesita para ejecutarse: solo vos, mientras diseñás.

¿POR QUÉ SIRVE UN DESIGNER «DE PYSIDE6» PARA PYQT6?

PyQt6 y PySide6 son dos puentes distintos hacia la **misma** biblioteca: Qt, escrita en C++. El `.ui` es un formato de Qt, no de PyQt. Designer lo escribe; PyQt6 lo lee. Instalamos `PySide6-Essentials` únicamente porque trae Designer listo para abrir con un comando, en Windows y en Mac.

¿Y PARA QUÉ UN ENTORNO VIRTUAL?

Un **entorno virtual** (`.venv`) es una carpeta con su propio Python y sus propias librerías, solo para este proyecto. Lo que instales ahí no rompe otros proyectos, y lo que necesita este proyecto queda anotado en `requirements.txt` para que cualquiera lo reconstruya. La carpeta `.venv` **no se sube a Git**: se vuelve a crear.

PASO A PASO

1 · PYTHON	Comprobá la versión. Necesitás **Python 3.10 o más nuevo**. Windows: `py --version` (si no anda, `python --version`) · Mac: `python3 --version`
2 · CARPETA	Creá la carpeta del proyecto y entrá en ella. Desde acá, **todos** los comandos se ejecutan parados en esta carpeta. `mkdir login-registro` y después `cd login-registro`
3 · ENTORNO	Creá el entorno virtual. Windows: `py -m venv .venv` · Mac: `python3 -m venv .venv`
4 · ACTIVAR	Activalo. Se hace **en cada terminal nueva**; la señal de que funcionó es que la línea empieza con `(.venv)`. Windows, PowerShell: `.venv\Scripts\Activate.ps1` · Windows, CMD: `.venv\Scripts\activate.bat` Mac (zsh o bash): `source .venv/bin/activate`

5 · INSTALAR	Con el entorno activo, el comando es **igual en los dos sistemas** (adentro del entorno, `python` ya apunta al Python correcto):
	`python -m pip install --upgrade pip`
	`python -m pip install PyQt6 PySide6-Essentials`
	Son unos 200 MB: con el Wi-Fi del aula puede tardar.

6 · DESIGNER	Abrilo con `pyside6-designer` (con el entorno activo). La primera vez puede tardar unos segundos. Tiene que aparecer el diálogo **New Form**.
7 · PYQT	Comprobá que PyQt6 se importa: `python -c "from PyQt6 import uic; print('PyQt6 OK')"`

MAC CON CHIP APPLE (M1, M2, M3, M4)

Los paquetes de PyQt6 y PySide6 son **universales**: corren nativos, sin Rosetta. No hace falta Homebrew ni descargar nada de qt.io, y **no busques Designer en Aplicaciones**: vive adentro de tu `.venv` y se abre con `pyside6-designer`. Si una guía de internet te da una ruta tipo `/Applications/Qt/…` o `/usr/local/…`, no es la tuya.

ALTERNATIVA, SOLO SI LO ANTERIOR NO FUNCIONA EN TU COMPU

Si no podés instalar con `pip` (por ejemplo, en una computadora del laboratorio sin permisos), instalá **Qt Creator** con el instalador oficial de código abierto de Qt (qt.io). Trae Designer integrado: abrís el `.ui` y se edita en el modo *Design*. Es una descarga bastante más grande y pide crear una cuenta de Qt.

La ventana de prueba

`prueba_entorno.py`

```python
import sys
from PyQt6.QtWidgets import QApplication, QLabel

app = QApplication(sys.argv)
etiqueta = QLabel("Si ves esto, PyQt6 funciona.")
etiqueta.resize(320, 80)
etiqueta.show()
sys.exit(app.exec())
```

Ejecutalo con `python prueba_entorno.py`. Es la misma estructura que el primer experimento de la guía de PyQt. Cuando funcione, podés borrarlo: no forma parte del proyecto.

PROBÁ · QUE VS CODE USE TU ENTORNO

Abrí la carpeta `login-registro` en VS Code. **Cmd/Ctrl + Shift + P** → «Python: Select Interpreter» → elegí el que dice `.venv`. Si te salteás este paso, el botón ▶ de VS Code ejecuta con otro Python y vas a ver `ModuleNotFoundError: No module named 'PyQt6'` aunque lo hayas instalado.

El `.gitignore`, ahora

`.gitignore`

```
# entorno virtual: se reconstruye con requirements.txt
.venv/

# archivos que genera Python solo
__pycache__/
*.pyc

# del sistema operativo
.DS_Store
Thumbs.db

# secretos (para cuando llegue la base de datos)
.env
```

Crealo **antes** del primer commit. Lo que entra al historial de Git no sale fácil. Es el mismo criterio de la guía de MySQL.

Errores frecuentes

LO QUE VES	QUÉ SIGNIFICA	CÓMO SALIR
`"python" no se reconoce como un comando…` (Windows)	Python no está en el PATH.	Usá `py`. Si tampoco anda, reinstalá Python tildando *Add python.exe to PATH*.
`zsh: command not found: python` (Mac)	En Mac el comando es `python3` … hasta que activás el entorno.	Usá `python3` para crear el entorno; después de activarlo, `python` funciona.
PowerShell: *la ejecución de scripts está deshabilitada en este sistema*	Windows bloquea el script de activación.	Usá CMD con `activate.bat`, o habilitalo solo para esa ventana: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`.
`pyside6-designer: command not found` / *no se reconoce*	El entorno no está activo, o la instalación no terminó.	¿Ves `(.venv)`? Activalo y repetí el paso 5.
`ModuleNotFoundError: No module named 'PyQt6'`	Estás ejecutando con un Python que no es el del entorno.	Activá el entorno en esa terminal o elegí el intérprete `.venv` en VS Code.
`ERROR: Could not find a version that satisfies the requirement…`	Tu Python es demasiado viejo (o de 32 bits).	Instalá Python 3.10 o más nuevo, de 64 bits, y recreá el entorno.

PENSÁ

- Si mañana borrás la carpeta `.venv`, ¿qué perdés? ¿Qué archivo te permite recuperarlo todo?
- ¿Por qué conviene escribir `python -m pip` en vez de `pip` a secas?

INVESTIGÁ

- PyQt6 y PySide6: ¿quién desarrolla cada una y con qué licencia? ¿Por qué su código se parece tanto?
- Ejecutá `python -c "import PyQt6; print(PyQt6.__file__)"` con el entorno activo y sin él. ¿Qué cambia? ¿Qué te dice eso sobre dónde «vive» una librería?

COMPROBÁ · NO AVANCES SIN ESTO

- [ ] La terminal muestra `(.venv)` al principio de la línea.
- [ ] `pyside6-designer` abre Qt Designer y aparece el diálogo **New Form**.
- [ ] `prueba_entorno.py` abre una ventana con el mensaje.
- [ ] VS Code usa el intérprete de `.venv`.

- [ ] Existe `.gitignore` y tiene `.venv/`.

03

Aprender Qt Designer explorando

Construir las dos pantallas sin escribir todavía una línea de lógica

NIVEL 1

EL PROBLEMA

En la guía de PyQt cada widget era una línea de código, cada posición una cuenta, y para ver cómo quedaba había que ejecutar. Con diez widgets y dos pantallas, diseñar así es lento y se rompe con cualquier cambio.

LA IDEA

QUÉ ES UN ARCHIVO .UI

Una **descripción de la pantalla**, en texto (formato XML): qué widgets hay, cómo se llama cada uno, qué propiedades tiene y cómo están organizados. Designer lo escribe mientras dibujás. **No contiene comportamiento**: no dice qué pasa cuando hacés clic.

```
<widget class="QPushButton" name="btn_login">
 <property name="text">
  <string>Iniciar sesión</string>
 </property>
</widget>
```

Un pedazo real de un `.ui`. `class` es qué widget es, `name` es su **objectName** y `text` es lo que se ve. Buscá la línea que dice qué hace al hacer clic: no existe. Nunca lo vas a editar a mano: para eso está Designer.

Las cinco zonas de Designer

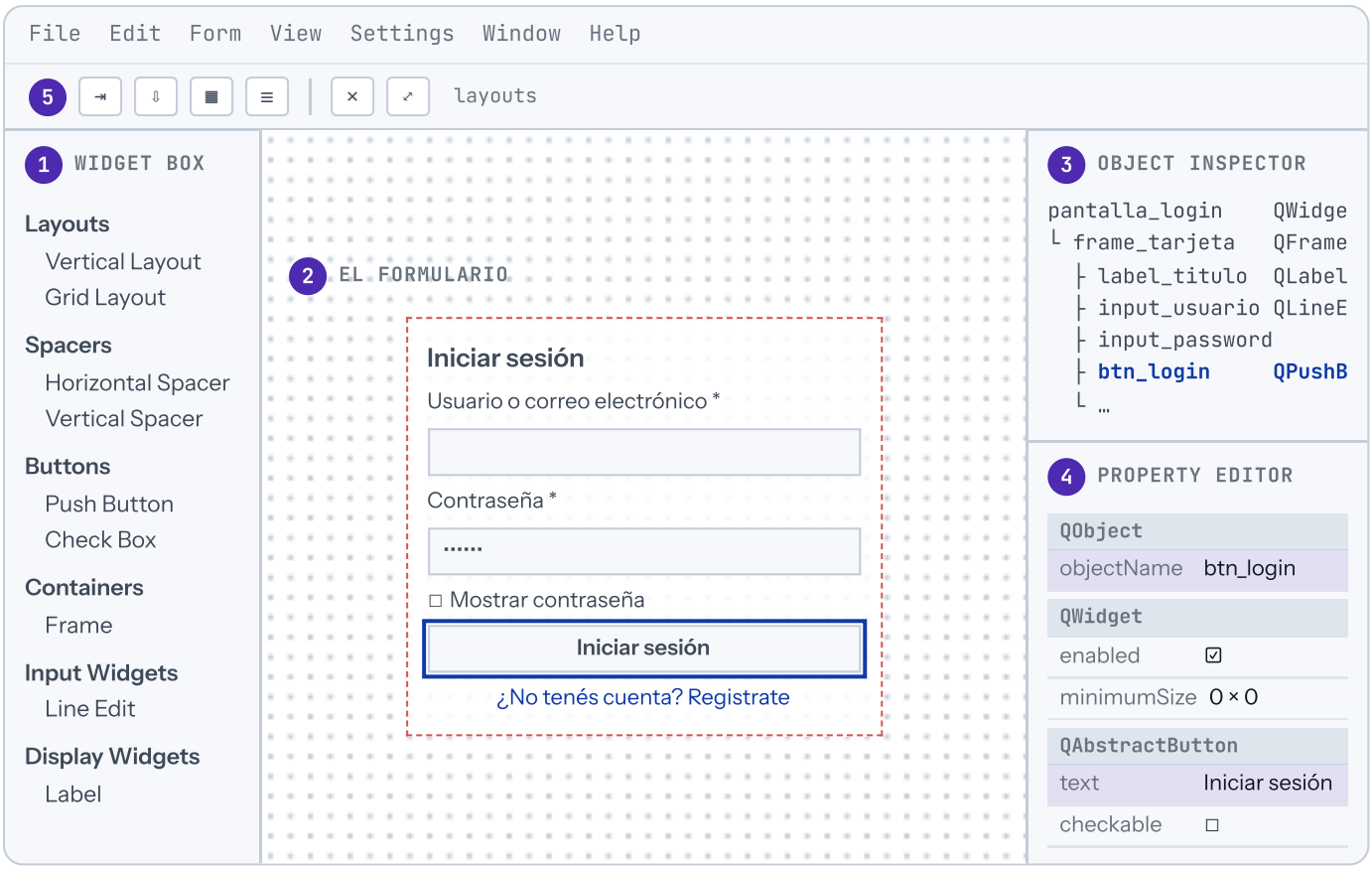


ILUSTRACIÓN CONCEPTUAL — esquema dibujado para esta guía, **no una captura**. La ubicación real de los paneles cambia según la versión y el sistema. Si cerrás un panel sin querer, se vuelve a abrir desde el menú *View* (en algunas versiones, *Window*). Designer suele estar en inglés: la guía usa sus nombres originales.

ZONA	QUÉ ES	QUÉ PROBLEMA RESUELVE
1 · Widget Box	El catálogo de piezas, por categoría.	No tenés que recordar el nombre de cada clase: lo arrastrás al formulario.
2 · Formulario	La pantalla que estás diseñando.	Ves el resultado mientras lo construís, sin ejecutar nada.
3 · Object Inspector	El árbol de quién está adentro de quién, con su nombre y su clase.	Seleccionar widgets aunque estén tapados, y ver si alguno quedó fuera de un layout.
4 · Property Editor	Todas las propiedades del widget seleccionado.	Configurar nombre, texto, tamaños y modos sin escribir código.
5 · Barra de layouts	Botones para organizar: vertical, horizontal, grilla, romper layout.	Que las posiciones se calculen solas, como en la sección 7 de la guía de PyQt.

MIRÁ BIEN EL PROPERTY EDITOR

Las propiedades están agrupadas en bloques: `QObject`, `QWidget`, `QAbstractButton`, `QPushButton`. Cada bloque es una clase padre. Un botón tiene `objectName` porque **es un** `QObject`, y tiene `text` porque **es un** `QAbstractButton`. Es la herencia de la clase 2, a la vista.

Los nombres que usa toda la guía

El código de las etapas siguientes depende de estos `objectName`. Si cambiás uno, tenés que cambiarlo también en Python: elegí respetarlos. La convención es **prefijo según el tipo + qué es**, en minúscula y con guion bajo.

LOGIN.UI	CLASE
`pantalla_login`	QWidget (el formulario)
`frame_tarjeta`	QFrame
`label_titulo`	QLabel
`input_usuario`	QLineEdit
`input_password`	QLineEdit
`chk_mostrar_password`	QCheckBox
`btn_login`	QPushButton
`label_mensaje`	QLabel (empieza vacío)
`btn_ir_registro`	QPushButton

REGISTRO.UI	CLASE
`pantalla_registro`	QWidget (el formulario)
`frame_tarjeta`	QFrame
`input_nombre`	QLineEdit
`input_apellido`	QLineEdit
`input_usuario`	QLineEdit
`input_password`	QLineEdit
`input_confirmar_password`	QLineEdit
`chk_mostrar_password`	QCheckBox
`btn_registrar`	QPushButton
`label_mensaje`	QLabel (empieza vacío)
`btn_volver_login`	QPushButton

Las etiquetas que acompañan a cada campo (`label_usuario`, `label_password`…) no las usa el código, pero nombralas igual: cuando el Object Inspector tiene veinte `label_3`, nadie encuentra nada. Y sí: `input_usuario` se repite en las dos pantallas. En la etapa 5 vas a ver por qué no chocan.

LA DEMOSTRACIÓN · LOGIN.UI, PASO A PASO

1 · NUEVO	*File* → *New…* → plantilla **Widget** → *Create*. **No elijas *Main Window***: trae barra de menú y de estado que no vamos a usar, y no encaja con la clase que vamos a escribir (en la etapa 5 vas a ver el error que produce).
2 · GUARDAR	Guardalo **ya**, vacío: *File* → *Save As* → entrá a `login-registro`, creá la carpeta `ventanas` y guardá como `login.ui`. Guardar al principio evita el clásico «lo guardé en Descargas». Después, Cmd/Ctrl + S seguido.
3 · NOMBRE	Con el fondo del formulario seleccionado, en el Property Editor cambiá `objectName` a `pantalla_login`.
4 · TARJETA	Arrastrá un **Frame** (en *Containers*) al centro del formulario. Nombralo `frame_tarjeta`. Todo lo demás va a vivir **adentro** de este frame.

5 · WIDGETS	Soltá dentro del frame, más o menos en columna: un **Label** (título), un Label y un **Line Edit** (usuario), un Label y un Line Edit (contraseña), un **Check Box**, un **Push Button**, un Label (mensaje) y otro Push Button (ir a registro). **No importa que queden torcidos**: el layout los va a alinear.
6 · TEXTOS	Doble clic sobre cada widget para escribir su texto visible. Al Label del mensaje **borrale el texto** (*TextLabel*): empieza vacío y lo llena Python.
7 · NOMBRES	Uno por uno, poné los `objectName` de la tabla. Revisalos en el Object Inspector.
8 · LAYOUT ADENTRO	Clic derecho en una zona vacía del frame → *Lay out* → *Lay Out Vertically*. Todo queda en columna.
9 · CENTRAR	Arrastrá cuatro espaciadores alrededor de la tarjeta: **Vertical Spacer** arriba y abajo, **Horizontal Spacer** a izquierda y derecha. Clic en el fondo del formulario → *Lay out* → *Lay Out in a Grid*. Los espaciadores empujan desde los cuatro lados y la tarjeta queda en el centro.
10 · CONTRASEÑA	Seleccioná `input_password` → propiedad `echoMode` → **Password**. De paso, completá `placeholderText` en los dos campos (el texto gris de ayuda).
11 · TAMAÑOS	En `frame_tarjeta`: `maximumSize` → ancho **400**, y `minimumSize` → ancho **360**. Así, con la ventana maximizada, la tarjeta no se estira de punta a punta.
12 · TAB	*Edit* → *Edit Tab Order*: hacé clic en los widgets en el orden en que una persona los recorre. *Esc* para salir.
13 · PREVIEW	*Form* → *Preview* (Ctrl + R en Windows, Cmd + R en Mac). Es tu pantalla funcionando, sin Python: escribí, apretá Tab, agrandá y achicá la ventana.

> **¿POR QUÉ UNA GRILLA Y NO UN LAYOUT VERTICAL?**
>
> Un layout vertical sobre el formulario centra la tarjeta de arriba a abajo, pero **no de izquierda a derecha**: con ancho máximo, la tarjeta queda pegada a un costado. Lo comprobamos al preparar esta guía. Con la grilla y los cuatro espaciadores se resuelven las dos direcciones a la vez, sin tocar código.

Cómo detectar errores de diseño

SEÑAL	QUÉ PASÓ	CÓMO SE ARREGLA
En el Object Inspector, un widget con un ícono rojo tachado	Ese contenedor no tiene layout: sus hijos están a mano.	Seleccioná el contenedor → *Lay out*.
En Preview, al agrandar, algo queda quieto o se superpone	Ese widget quedó afuera del layout.	Arrastralo adentro del layout (aparece una línea azul donde va a caer).
Nombres como `lineEdit_2` o `pushButton`	Te faltó poner el `objectName`.	Renombralo con la tabla.
Un texto aparece cortado: «Usuario o corre…»	Un tamaño fijo es más chico que el texto.	Sacá el tamaño fijo o activá `wordWrap` en el Label.
Al reacomodar, todo se desarma	Se rompió un layout.	*Break Layout* sobre el contenedor y volvé a aplicarlo.

AHORA SIN PASOS · REGISTRO.UI

Diseñá `ventanas/registro.ui` con la misma técnica y la tabla de nombres. Nadie te va a dictar los clics: ya sabés hacerlo.

- Nombre y apellido pueden ir lado a lado: armá dos columnas (Label + Line Edit) y aplicales *Lay Out Horizontally*.
- Los dos campos de contraseña, con `echoMode` en **Password**.
- Podés copiar la tarjeta de `login.ui` y pegarla como punto de partida, pero **revisá todos los nombres**: Designer agrega sufijos como `_2` cuando un nombre ya existe.

PROBÁ · TEXTO VISIBLE ≠ OBJECTNAME

Seleccioná el botón de login. En el formulario dice **Iniciar sesión**; en el Object Inspector dice **btn_login**. Son dos propiedades distintas, para dos lectores distintos. Completá:

WIDGET	TEXTO VISIBLE (LO LEE…)	OBJECTNAME (LO LEE…)
Botón principal	Iniciar sesión — *la persona*	btn_login — *el código*
Campo de contraseña		
Casilla para mostrar		
Enlace a registro		

- En Preview, cambiá el `echoMode` de la contraseña a *Normal*, *NoEcho* y *PasswordEchoOnEdit*. ¿Qué hace cada uno? Volvé a dejarlo en *Password*.

PENSÁ

Si cambiás el texto del botón a «Entrar», ¿hay que tocar Python? ¿Y si cambiás su `objectName` a «boton_entrar»?

¿Por qué `label_mensaje` existe vacío desde el diseño, en lugar de crearse desde Python cuando hace falta?

INVESTIGÁ

- ¿Qué es la propiedad **buddy** de un `QLabel`? Probá *Edit* → *Edit Buddies*, poné un `&` en el texto de una etiqueta («&Usuario») y usá Alt + U en Preview. ¿Para qué sirve, y a quién ayuda?
- ¿En qué se diferencia un **Form Layout** de un layout vertical para pares etiqueta–campo?

COMPROBÁ

- [ ] `login.ui` y `registro.ui` están en `login-registro/ventanas/`.
- [ ] En Preview, al agrandar la ventana, la tarjeta queda centrada y nada se superpone.
- [ ] Ningún contenedor muestra el ícono de «sin layout» en el Object Inspector.
- [ ] Todos los `objectName` de las tablas están puestos, sin `_2` ni mayúsculas distintas.
- [ ] Las contraseñas se ven con puntos y Tab recorre los campos en orden.

04

De `.ui` a Python

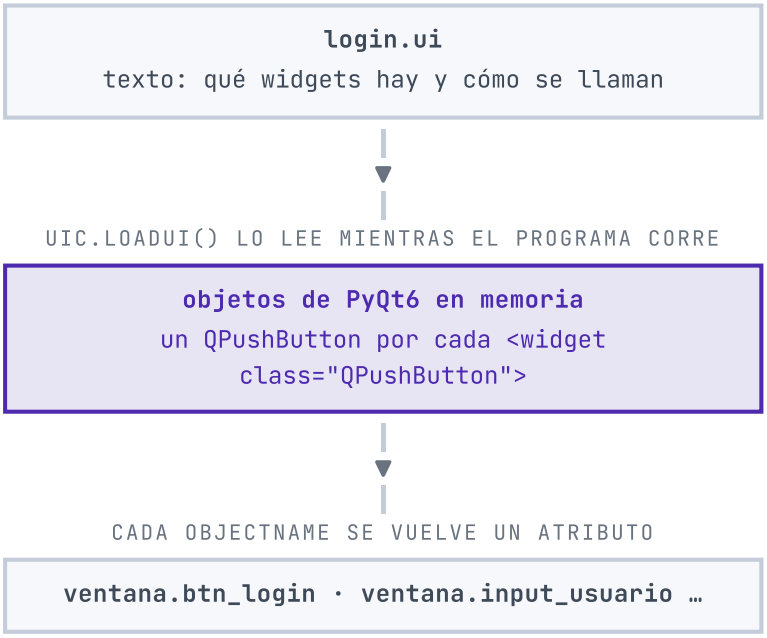
Que tu programa lea el diseño y lo convierta en una ventana de verdad

NIVEL 1

EL PROBLEMA

Tenés un `login.ui` impecable. Python no sabe que existe.

LA IDEA



LA DEMOSTRACIÓN

`prueba_ui.py` (en la carpeta login-registro)

```python
import sys

from PyQt6 import uic
from PyQt6.QtWidgets import QApplication

app = QApplication(sys.argv)

ventana = uic.loadUi("ventanas/login.ui")
ventana.show()

sys.exit(app.exec())
```

Ejecutalo con `python prueba_ui.py`, parado en la carpeta del proyecto. Tu diseño, ahora como programa. Es un archivo de prueba: en la etapa 5 lo reemplazamos por una clase.

Línea por línea

`QAPPLICATION`	La **aplicación**. Va antes que cualquier widget y hay **una sola** por programa. Sin ella: `QWidget: Must construct a QApplication before a QWidget`, y el programa se cierra.
`UIC.LOADUI()`	Abre el `.ui`, lee el XML y **crea todos los widgets** con sus propiedades y layouts. Devuelve el widget raíz: el formulario.
`VENTANA`	Ese widget raíz. Probá `type(ventana)`: es un `QWidget` común, porque elegiste la plantilla *Widget*.
`SHOW()`	Crear no es mostrar. Sin `show()` la ventana existe en memoria y nadie la ve. No da error: simplemente no aparece nada.
`APP.EXEC()`	Arranca el **bucle de eventos** y no termina hasta que cerrás la ventana. `sys.exit()` le pasa al sistema operativo el código con el que terminó.

El bucle de eventos

ESPERA	→ EVENTO	→ RESPONDE	→ REDIBUJA
No hace nada. Espera a que pase algo.	Un clic, una tecla, la ventana que cambia de tamaño.	Le avisa al widget, y si conectaste un método, lo ejecuta.	Actualiza lo que cambió en pantalla.

↺ vuelve a esperar · así hasta que cerrás la ventana · recién ahí `exec()` termina

Es la misma idea del `while` del menú de Arena de Combate, pero ya escrito por Qt. Por eso un programa con interfaz no «termina» después de la última línea: se queda dando vueltas acá adentro.

Acceder a los widgets por su nombre

```python
print(ventana.btn_login.text())                # lee el texto visible
ventana.label_mensaje.setText("Hola desde Python")
print(ventana.input_usuario.placeholderText())
```

Agregá estas líneas antes de `ventana.show()`. **El nombre del atributo es el objectName**: si en Designer dice `btn_login`, en Python es `ventana.btn_login`. El objectName es el puente entre los dos mundos.

La ruta del archivo

PROBÁ · ROMPELO A PROPÓSITO

Salí de la carpeta del proyecto (`cd ..`) y ejecutá `python login-registro/prueba_ui.py` . Leé lo que aparece.

```
Traceback (most recent call last):
  File "…/login-registro/prueba_ui.py", line 8, in <module>
    ventana = uic.loadUi("ventanas/login.ui")
  File "…/.venv/lib/…/PyQt6/uic/load_ui.py", line …, in loadUi
  …   (varias líneas internas de PyQt6)   …
PyQt6.uic.exceptions.UIFileException: ventanas/login.ui: invalid Qt Designer file: [Errno 2] No such file or directory: 'ventanas/login.ui'
```

1. Un traceback se lee **de abajo hacia arriba**. La última línea dice **qué** pasó: el tipo de error y el mensaje.
2. No te quedes con la primera mitad del mensaje. «invalid Qt Designer file» confunde: el archivo no es inválido, **no existe** en esa ruta. La causa real está al final: `No such file or directory` .
3. Las líneas `File …` dicen **dónde**. Buscá la primera que sea un archivo tuyo (la resaltada): ahí está la línea que lo provocó. Las de adentro de `.venv` son de la librería; casi nunca el problema está ahí.

POR QUÉ PASÓ

Una ruta relativa como `"ventanas/login.ui"` se calcula **desde la carpeta en la que estás parado en la terminal**, no desde donde está el archivo `.py` . VS Code, según cómo lo ejecutes, puede pararse en otra carpeta. La solución es construir la ruta a partir de la ubicación del propio archivo:

```python
from pathlib import Path

RUTA_UI = Path(__file__).parent / "ventanas" / "login.ui"
print(RUTA_UI, RUTA_UI.exists())      # para comprobar, después lo sacás

ventana = uic.loadUi(RUTA_UI)
```

`__file__` es la ruta de este `.py` ; `.parent` , la carpeta que lo contiene; la `/` une partes de una ruta y funciona igual en Windows y en Mac. Ahora el programa encuentra el `.ui` desde donde sea que lo ejecutes.

Errores habituales y cómo diagnosticarlos

Todos estos mensajes son reales: los provocamos a propósito con PyQt6 al preparar esta guía.

ÚLTIMA LÍNEA DEL ERROR	CAUSA	QUÉ HACER
`UIFileException: …: invalid Qt Designer file: [Errno 2] No such file or directory`	La ruta al `.ui` no existe desde donde ejecutaste.	Usá `Path(__file__).parent` e imprimí `RUTA_UI.exists()` .
`UIFileException: …: invalid Qt Designer file: no element found: line 1, column 0`	El `.ui` está vacío o roto (se guardó a medias, o alguien lo editó a mano).	Abrilo en Designer y guardalo de nuevo. Si no abre, recuperalo de Git.
`AttributeError: 'QWidget' object has no attribute 'boton_login'. Did you mean: 'btn_login'?`	Usaste un nombre que no es el `objectName` .	Mirá el Object Inspector. Y leé el final: Python te sugiere el nombre correcto.

ÚLTIMA LÍNEA DEL ERROR	CAUSA	QUÉ HACER
`ModuleNotFoundError: No module named 'PyQt6'`	Python equivocado.	Activá el entorno o elegí el intérprete `.venv`.
`QWidget: Must construct a QApplication before a QWidget`	Creaste o cargaste un widget antes de `QApplication`.	`app = QApplication(sys.argv)` va primero.
No aparece nada y no hay error	Falta `show()`.	Agregalo antes de `app.exec()`.
La ventana aparece y desaparece al instante	Falta `app.exec()`: el programa llegó al final y terminó.	`sys.exit(app.exec())` como última línea.

PROBÁ

- Escribí un texto en `label_mensaje` desde Python y miralo aparecer.
- Listá todos los botones del formulario: `for boton in ventana.findChildren(QPushButton): print(boton.objectName())` (acordate de importar `QPushButton`).
- En Designer, cambiá el texto del botón principal y guardá. Volvé a ejecutar **sin tocar Python**.

PENSÁ

- ¿Por qué el cambio del último punto aparece sin tocar Python? ¿Qué habría pasado si hubieras convertido el `.ui` a `.py` con `pyuic6`?
- ¿Qué devuelve `ventana.objectName()`? ¿De dónde sale ese valor?

BITÁCORA DE ERRORES

Anotá el primer error real que te apareció (no los provocados): qué decía la última línea, qué creíste al principio y qué era en realidad.

COMPROBÁ

- [ ] `prueba_ui.py` muestra tu login.
- [ ] Funciona aunque lo ejecutes desde otra carpeta (ruta con `Path`).
- [ ] Cambiaste un texto desde Python y lo viste en la ventana.
- [ ] Puedo explicar qué hacen `QApplication`, `loadUi()`, `show()` y `exec()`.
- [ ] Sé encontrar en un traceback la línea de mi código que provocó el error.

05

Aplicar POO

Una clase propia para cada pantalla

NIVEL 1

EL PROBLEMA

`prueba_ui.py` anda, pero todo está suelto: una variable `ventana` global y, cuando haya que responder a cinco controles, cinco funciones sueltas tocando esa global. Con dos pantallas, el doble. Es exactamente el problema que cerró la sección 8 de la guía de PyQt.

LA IDEA

UNA PANTALLA ES UN OBJETO

Tiene estado (lo que hay escrito, qué mensaje se ve) y comportamiento (iniciar sesión, mostrar la contraseña). Entonces la escribimos como una **clase**: `VentanaLogin`. El diseño sigue en `login.ui`; la clase lo carga y le agrega lo que el diseño no puede tener: **qué hacer**.

```
uic.loadUi(RUTA_UI)          # crea un widget nuevo y te lo devuelve
uic.loadUi(RUTA_UI, self)    # «llená ESTE objeto» con los widgets del .ui
```

El segundo argumento cambia todo: los widgets se convierten en atributos de **nuestro** objeto: `self.btn_login`, `self.input_usuario`… Para que eso funcione, `self` tiene que ser el mismo tipo de widget que la raíz del `.ui`: un `QWidget`.

HERENCIA · ES UNA PANTALLA

`class VentanaLogin(QWidget)` y `uic.loadUi(RUTA_UI, self)`. La clase **es** el widget: se puede meter directamente adentro de otra ventana (lo vas a necesitar en la etapa 6) y escribís `self.btn_login`. **Es la que usamos.**

COMPOSICIÓN · TIENE UNA INTERFAZ

`class VentanaLogin:` sin padre, y adentro `self.ui = uic.loadUi(RUTA_UI)`. También funciona: escribís `self.ui.btn_login` y, para mostrarla, `self.ui.show()`. Es válida, y es la misma discusión ES UN / TIENE UN de la clase 2.

La herencia acá no está para «demostrar que existe»: la pide `loadUi(RUTA_UI, self)`, y la pide la etapa 6, donde cada pantalla tiene que poder entrar en otra ventana como un widget más.

LA DEMOSTRACIÓN

ventanas/ventana_login.py

```python
from pathlib import Path

from PyQt6 import uic
from PyQt6.QtWidgets import QWidget

RUTA_UI = Path(__file__).parent / "login.ui"


class VentanaLogin(QWidget):
    """Controla la pantalla de inicio de sesión. El diseño vive en login.ui."""

    def __init__(self):
        super().__init__()
        uic.loadUi(RUTA_UI, self)
        self._conectar_eventos()

    def _conectar_eventos(self):
        self.btn_login.clicked.connect(self.iniciar_sesion)

    def iniciar_sesion(self):
        self.label_mensaje.setText("Hiciste clic en Iniciar sesión.")
```

main.py

```python
import sys

from PyQt6.QtWidgets import QApplication

from ventanas.ventana_login import VentanaLogin


def main():
    app = QApplication(sys.argv)
    ventana = VentanaLogin()
    ventana.show()
    sys.exit(app.exec())


if __name__ == "__main__":
    main()
```

Ejecutá `python main.py` desde la carpeta del proyecto. `RUTA_UI` ahora es `"login.ui"` a secas porque `ventana_login.py` vive en la misma carpeta que el diseño. Ya podés borrar `prueba_ui.py`.

EN EL CÓDIGO	ES LO QUE YA SABÍAS
`class VentanaLogin(QWidget)`	Herencia: nuestra pantalla ES UN `QWidget`, más lo nuestro.
`super().__init__()`	Que el padre se arme primero, como en `Jugador`. Sin esto, el objeto no es todavía un widget.
`uic.loadUi(RUTA_UI, self)`	Llena `self` con los widgets del diseño. Desde esta línea existe `self.btn_login`.

EN EL CÓDIGO	ES LO QUE YA SABÍAS
`self._conectar_eventos()`	Un método propio con una sola tarea. El `_` avisa «uso interno de la clase»: encapsulación por convención.
`def iniciar_sesion(self)`	Un método público: la acción de esta pantalla. Por ahora solo cambia un texto.
`RUTA_UI`	Una constante del módulo: en mayúsculas porque no cambia, fuera de la clase porque no depende de ningún objeto.
`if __name__ == "__main__":`	Arranca la app solo si ejecutás `main.py`, no si alguien lo importa.

El diseño y el código, lado a lado

VENTANAS/LOGIN.UI · CÓMO ES

```
<widget class="QPushButton"
name="btn_login">
 <property name="text">
  <string>Iniciar sesión</string>
 </property>
</widget>
```

⇄ objectName btn_login

VENTANAS/VENTANA_LOGIN.PY · QUÉ HACE

```
self.btn_login.clicked.connect(
    self.iniciar_sesion
)
```

A la izquierda nunca hay comportamiento. A la derecha nunca hay posiciones, tamaños ni colores. Lo único que comparten es el nombre.

PROBÁ · EL ORDEN DEL __INIT__ IMPORTA

Hacé cada cambio, ejecutá, leé la última línea del error y volvé a dejarlo bien:

- Poné `uic.loadUi(...)` **antes** de `super().__init__()` → `RuntimeError: super-class __init__() of type VentanaLogin was never called`
- Poné `self._conectar_eventos()` **antes** de `loadUi` → `AttributeError: 'VentanaLogin' object has no attribute 'btn_login'`
- Sacale el `self` a `def iniciar_sesion(self)` y hacé clic. ¿Qué dice el error?

Si en Designer elegiste *Main Window* en vez de *Widget*, vas a ver `TypeError: ('Wrong base class of toplevel widget', (…VentanaLogin…, 'QMainWindow'))`: el `.ui` es una ventana principal y la clase es un widget común. Rehacé el formulario con la plantilla *Widget*.

TU TURNO · VENTANAREGISTRO

Escribí `ventanas/ventana_registro.py` con la clase `VentanaRegistro`, con el mismo patrón. Por ahora, `btn_registrar` conectado a un método `registrar()` que solo escribe un texto en `label_mensaje`. Para probarla, cambiá temporalmente `main.py` para que muestre `VentanaRegistro()`.

PENSÁ

Las dos pantallas tienen un widget llamado `input_usuario`. ¿Chocan? ¿Por qué no?

¿En qué caso te parecería mejor la composición (`self.ui`) que la herencia?

INVESTIGÁ

¿Qué valor tiene `__name__` cuando ejecutás un archivo y cuándo lo importás? Probalo con un `print(__name__)` en `main.py` y en `ventana_login.py`. ¿Qué pasaría al importar `main` desde otro archivo si no existiera el `if __name__ == "__main__":`? (Es el mismo problema del desafío final de la guía de PyQt.)

COMPROBÁ

- [ ] `python main.py` abre el login desde `VentanaLogin`.
- [ ] El clic en `btn_login` cambia `label_mensaje`.
- [ ] `VentanaRegistro` existe y carga `registro.ui`.
- [ ] No hay ninguna variable global con widgets.

06

Conectar eventos

Cinco controles, cinco respuestas, y una ventana que cambia de pantalla

NIVEL 1 · CONEXIONES NIVEL 2 · NAVEGACIÓN Y CONTRASEÑA

EL PROBLEMA

Hay cinco cosas que una persona puede tocar y cada una necesita una respuesta. Si las conectás todas juntas y algo falla, no vas a saber cuál fue.

LA IDEA · SEÑALES Y SLOTS

```
self.btn_login.clicked.connect(self.iniciar_sesion)
```

`self.btn_login`
El **emisor**: el botón que creó `loadUi` a partir del objectName.

`.clicked`
La **señal**: algo que *puede* pasar. No pasa ahora; puede pasar en cualquier momento.

`.connect( … )`
La **conexión**: «cuando pase, avisale a…». Se hace una sola vez, en el `__init__`.

`self.iniciar_sesion`
El **slot**: el método que responde. **Sin paréntesis**: lo entregás, no lo ejecutás.

EL ERROR MÁS COMÚN, OTRA VEZ

Con `connect(self.iniciar_sesion())` Python ejecuta el método **en ese momento** —al abrir la ventana— y le pasa a `connect` lo que devolvió: `None`. El resultado: `TypeError: argument 1 has unexpected type 'NoneType'`. Si ves `NoneType` en un error de `connect`, buscá paréntesis de más.

El contrato de este TP

WIDGET	SEÑAL	MÉTODO QUE RESPONDE	QUÉ RECIBE	NIVEL
`btn_login`	`clicked`	`VentanaLogin.iniciar_sesion`	nada	1
`btn_registrar`	`clicked`	`VentanaRegistro.registrar`	nada	1
`chk_mostrar_password` (en las dos)	`toggled`	`alternar_password`	`True` / `False`	2
`btn_ir_registro`	`clicked`	`VentanaPrincipal.mostrar_registro`	nada	2
`btn_volver_login`	`clicked`	`VentanaPrincipal.mostrar_login`	nada	2

EXPERIMENTOS, DE A UNO

1 · PRINT	Antes de escribir lógica, conectá cada control a un método que solo haga `print("clic en btn_login")`. Ejecutá, hacé clic y mirá **la terminal**, no la ventana. Si el texto aparece, la conexión está bien.
2 · UN DATO	`toggled` **trae un valor**: si la casilla quedó tildada o no. Conectala a `def alternar_password(self, visible): print("visible:", visible)` y tildá y destildá varias veces.
3 · ENTER	Conectá `self.input_password.returnPressed` también a `iniciar_sesion`. Ahora Enter en la contraseña equivale a hacer clic. Dos señales, un mismo slot.
4 · ROMPER	Ponele paréntesis a un slot dentro del `connect`. Leé el error. Sacáselos.

Mostrar u ocultar la contraseña

```
from PyQt6.QtWidgets import QLineEdit, QWidget     # QLineEdit se suma al import


class VentanaLogin(QWidget):
    ...

    def alternar_password(self, visible):
        if visible:
            self.input_password.setEchoMode(QLineEdit.EchoMode.Normal)
        else:
            self.input_password.setEchoMode(QLineEdit.EchoMode.Password)
```

En PyQt6 las opciones fijas van con su nombre completo: `QLineEdit.EchoMode.Password`, no `QLineEdit.Password` como en ejemplos viejos de PyQt5. En el registro hay **dos** campos de contraseña: resolvelo vos.

¿Quién cambia de pantalla?

? Si `VentanaLogin` crea una `VentanaRegistro` para mostrarla, y `VentanaRegistro` crea una `VentanaLogin` para volver... ¿cuántas ventanas hay después de ir y volver diez veces?

LA IDEA

Ninguna pantalla debería saber que la otra existe. Las conoce un **tercero**: la ventana de la aplicación, que tiene las dos y muestra una por vez. Para eso Qt trae `QStackedWidget`, **una pila de pantallas** —como un mazo de cartas donde solo se ve la de arriba—. Nuestra `VentanaPrincipal` **es una** pila (herencia) y **tiene** las dos pantallas (composición): igual que `Jugador`, que ES UN `Personaje` y TIENE UN `Arma`.

`ventanas/ventana_principal.py`

```
from PyQt6.QtWidgets import QStackedWidget

from ventanas.ventana_login import VentanaLogin
from ventanas.ventana_registro import VentanaRegistro


class VentanaPrincipal(QStackedWidget):
    """La ventana de la aplicación: muestra una pantalla por vez."""

    def __init__(self):
        super().__init__()
        self.login = VentanaLogin()
        self.registro = VentanaRegistro()
        self.addWidget(self.login)
        self.addWidget(self.registro)

        self.login.btn_ir_registro.clicked.connect(self.mostrar_registro)
        self.registro.btn_volver_login.clicked.connect(self.mostrar_login)

    def mostrar_login(self):
        ...  # investigá: ¿qué método de QStackedWidget elige la pantalla visible?

    def mostrar_registro(self):
        ...
```

En `main.py`, cambiá `VentanaLogin()` por `VentanaPrincipal()`. Los dos `...` son tuyos.

▼ ▸ PISTA PARA MOSTRAR_LOGIN Y MOSTRAR_REGISTRO

Buscá en la documentación de `QStackedWidget` los métodos que empiezan con `setCurrent`. Hay dos: uno recibe un número de posición y el otro recibe el widget. ¿Cuál se lee mejor dentro de un método llamado `mostrar_registro`?

¿Por qué `mostrar_registro()` no es un método de `VentanaLogin`? Porque cambiar de pantalla no es responsabilidad del login: el login sabe iniciar sesión. Quien sabe qué pantallas existen es `VentanaPrincipal`.

PENSÁ

- `VentanaPrincipal` toca `self.login.btn_ir_registro`, un widget de **otra** clase. ¿Te parece prolijo? ¿Qué pasaría si mañana el login tuviera dos formas de ir al registro? (Lo retoma el nivel 3.)
- ¿Por qué `toggled` le pasa un valor al método y `clicked` no necesita pasar nada?

INVESTIGÁ

¿Qué otras señales tiene un `QLineEdit`? Encontrá la diferencia entre `textChanged` y `textEdited`, y pensá para qué te serviría cada una en este TP.

COMPROBÁ

- [ ] Cada uno de los cinco controles hace algo visible (o imprime en la terminal).
- [ ] Enter en la contraseña equivale a hacer clic en **Iniciar sesión**.
- [ ] N2 Tildar muestra la contraseña; destildar la oculta. En el registro, las dos.
- [ ] N2 Se puede ir y volver entre login y registro todas las veces que quieras, en una sola ventana.

07

Validar los formularios

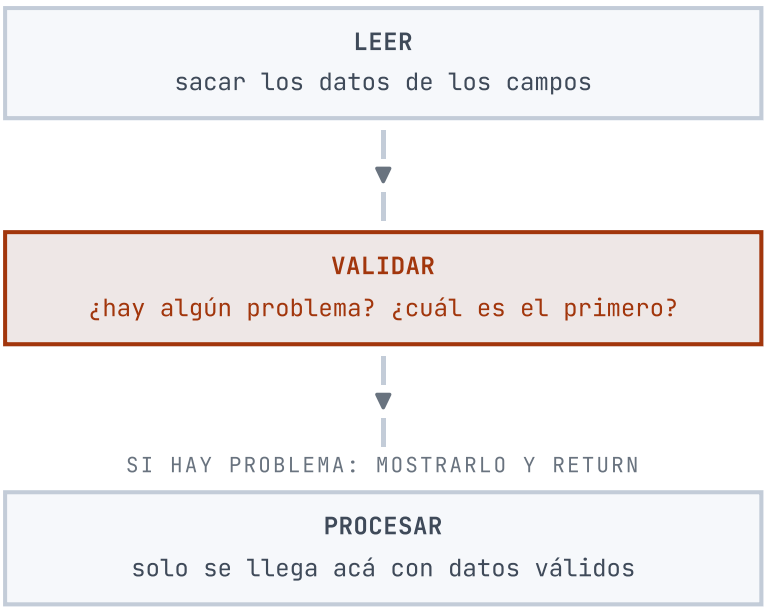
Antes de pensar en una base de datos, que los datos tengan sentido

NIVEL 1

EL PROBLEMA

Hoy, si apretás **Iniciar sesión** con todo vacío, el programa sigue como si nada. Y si mañana mostrara «¡Bienvenido!» con cualquier dato, alguien va a creer que entró… cuando nadie verificó nada.

LA IDEA · LEER, VALIDAR, CORTAR O SEGUIR



LA DEMOSTRACIÓN · EL LOGIN

ventanas/ventana_login.py (los métodos nuevos)

```
    def iniciar_sesion(self):
        usuario = self.input_usuario.text().strip()
        password = self.input_password.text()

        error = self._validar(usuario, password)
        if error:
            self._mostrar_mensaje(error)
            return

        self._mostrar_mensaje(
            "Datos completos. La autenticación todavía no está implementada."
        )

    def _validar(self, usuario, password):
        if not usuario:
            return "Ingresá tu usuario o correo electrónico."
        if not password:
            return "Ingresá tu contraseña."
        return None

    def _mostrar_mensaje(self, texto):
        self.label_mensaje.setText(texto)
```

`_VALIDAR`	Decide, no muestra. Devuelve **el texto del problema** o `None` si no hay ninguno. Una responsabilidad: «¿está bien esto?».
`IF ERROR`	Una **cláusula de guarda**: si hay problema, se muestra y el `return` corta el método ahí. Nada de lo que está abajo se ejecuta con datos inválidos.
`_MOSTRAR_MENSAJE`	El **único** lugar que toca `label_mensaje`. En la etapa 9 le vamos a sumar colores y vas a cambiar un solo método.
`EL ÚLTIMO TEXTO`	No dice «Bienvenido». Dice la verdad: los datos están completos, pero **nadie los verificó**.

TU TURNO · VALIDAR EL REGISTRO

Reglas: nombre, apellido y usuario obligatorios; contraseña de **al menos 8 caracteres**; contraseña y confirmación **iguales**. Si algo falla, **no se registra nada**.

La regla del mínimo no es de la pantalla: es del sistema. Creá `controladores/autenticacion.py` con una sola línea por ahora, `LONGITUD_MINIMA_PASSWORD = 8`, e importala desde la ventana. En la etapa 8 ese archivo crece.

ventanas/ventana_registro.py (esqueleto)

```
from controladores.autenticacion import LONGITUD_MINIMA_PASSWORD


class VentanaRegistro(QWidget):
    ...

    def registrar(self):
        datos = self._leer_formulario()
        error = self._validar(datos)
        if error:
            self._mostrar_mensaje(error)
            return
        self._mostrar_mensaje("Formulario válido (todavía no se guarda).")

    def _leer_formulario(self):
        return {
            "nombre": self.input_nombre.text().strip(),
            # completá: "apellido", "usuario", "password" y "confirmacion"
        }

    def _validar(self, datos):
        ...  # una regla por if, en el orden en que se completa el
formulario
        return None
```

PROBÁ · CASOS DE PRUEBA

Cargá cada caso y tildá si apareció **exactamente** el mensaje esperado (o uno equivalente con tus palabras). Se muestra un error por vez: el primero que encuentra.

CASO	DATOS	MENSAJE ESPERADO	¿PASÓ?
1	Todo vacío	Completá tu nombre.	☐
2	Nombre: tres espacios	Completá tu nombre.	☐
3	Nombre y apellido; usuario vacío	Completá tu usuario o correo electrónico.	☐

CASO	DATOS	MENSAJE ESPERADO	¿PASÓ?
4	Todo completo; contraseña `1234567` (7)	La contraseña necesita al menos 8 caracteres.	☐
5	Contraseña `12345678`, confirmación `12345679`	Las contraseñas no coinciden.	☐
6	Todo correcto	Formulario válido…	☐
7	Login: usuario con datos, contraseña vacía	Ingresá tu contraseña.	☐

PENSÁ

¿Por qué le aplicamos `strip()` al usuario y **no** a la contraseña?

¿Conviene mostrar un error por vez o todos juntos? ¿Qué es más claro para quien completa el formulario?

VALIDAR EL FORMULARIO NO ES ASEGURAR EL SISTEMA

La validación de la interfaz **ayuda a la persona**: le avisa rápido y en su idioma qué le falta. Pero corre en su computadora y se puede esquivar: otro programa, un script, una versión vieja de tu app. Por eso, cuando exista un servidor, **él va a tener que validar todo de nuevo**. Y ninguna de las dos cosas es **autenticar**: comprobar que alguien es quien dice ser.

INVESTIGÁ

¿Qué hacen `QLineEdit.setMaxLength()` y un `QRegularExpressionValidator`? ¿Reemplazan tu `_validar` o lo complementan? Y opcional: con `textChanged` y `setEnabled()`, ¿podrías deshabilitar el botón mientras falten datos?

COMPROBÁ

- ☐ Los siete casos de la tabla dan el mensaje esperado.
- ☐ Con datos inválidos nunca aparece un mensaje de éxito.
- ☐ El login nunca dice «bienvenido» sin haber verificado nada.
- ☐ El número 8 está escrito **una sola vez** en todo el proyecto.

08 Simular el funcionamiento

Un login que recuerda usuarios… mientras la app está abierta

NIVEL 1 · MENSAJES HONESTOS | NIVEL 2 · SERVICIO COMPARTIDO

✓ HECHO

Interfaz

Dos pantallas diseñadas, cargadas y conectadas.

✓ HECHO

Validaciones

Ningún dato sin sentido pasa del formulario.

◐ SIMULADO

Autenticación

Usuarios en memoria. **No** es un sistema real: se borra al cerrar.

EL PROBLEMA

Para que el login tenga sentido, **alguien tiene que acordarse** de quién se registró. Todavía no hay base de datos.

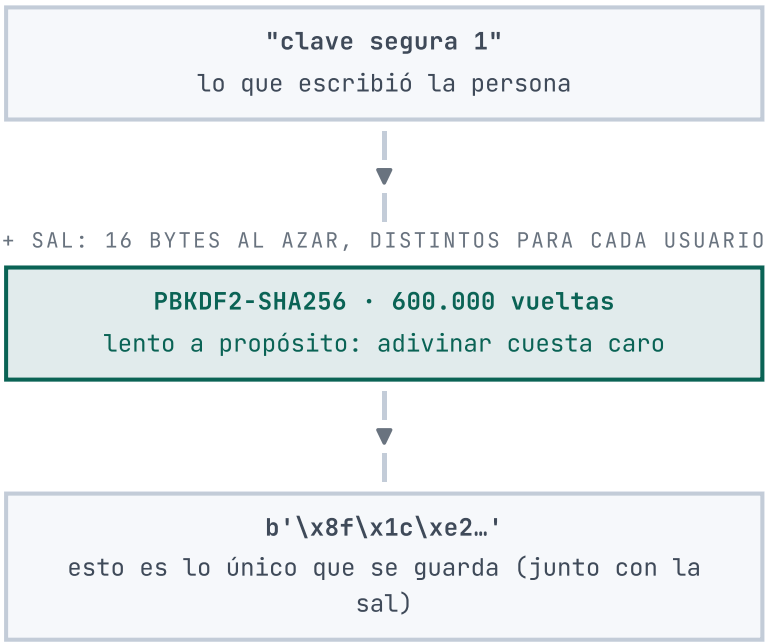
LA IDEA · UN SERVICIO QUE NO SABE QUE EXISTE UNA VENTANA

SERVICIOAUTENTICACION

Una clase en `controladores/autenticacion.py` con dos responsabilidades: **registrar** y **autenticar**. Guarda los usuarios en un diccionario. No importa nada de PyQt: no sabe que hay pantallas. Es la misma idea que `BasededatosControlador` en la guía de MySQL: **un lugar, una responsabilidad**.

NI SIQUIERA EN UNA SIMULACIÓN: CONTRASEÑAS EN TEXTO PLANO

Aunque esto viva en memoria, no guardamos la contraseña. Guardamos un **hash con sal**: el resultado de una cuenta que no se puede deshacer. Para verificar, se repite la cuenta con lo que escribió la persona y se comparan los resultados. Así, quien mire los datos —vos, un compañero, alguien que los robe— no ve ninguna contraseña. Y como la gente repite contraseñas, filtrar una no compromete sus otras cuentas.



LA DEMOSTRACIÓN

controladores/autenticacion.py

```python
"""Servicio de autenticación SIMULADO, en memoria.

No es un sistema de producción: los usuarios se pierden al cerrar la app,
no hay límite de intentos, ni sesiones, ni base de datos.
Lo que sí hace bien desde ahora: nunca guarda la contraseña, solo su hash con sal.
"""
import hashlib
import hmac
import os

LONGITUD_MINIMA_PASSWORD = 8
ITERACIONES = 600_000


class ServicioAutenticacion:

    def __init__(self):
        # "ana@correo.com" → {"nombre": ..., "sal": ..., "hash": ...}
        self._usuarios = {}

    def registrar(self, nombre, apellido, usuario, password):
        clave = usuario.strip().lower()
        minimo = LONGITUD_MINIMA_PASSWORD
        if len(password) < minimo:
            raise ValueError(f"La contraseña necesita al menos {minimo} caracteres.")
        if clave in self._usuarios:
            raise ValueError("Ese usuario ya está registrado.")
        # completá: ¿qué otra regla tendría que revisar el servicio,
        # aunque la pantalla ya la revise?

        sal = os.urandom(16)
        self._usuarios[clave] = {
            "nombre": nombre.strip(),
            "apellido": apellido.strip(),
            "sal": sal,
            "hash": self._calcular_hash(password, sal),
        }

    def autenticar(self, usuario, password):
        ...  # completá: True solo si el usuario existe y la contraseña coincide

    @staticmethod
    def _calcular_hash(password, sal):
        return hashlib.pbkdf2_hmac(
            "sha256", password.encode("utf-8"), sal, ITERACIONES
        )
```

El cálculo del hash está completo a propósito: **la criptografía no se inventa**, se usa la que ya existe. Lo que te toca escribir es la lógica: qué revisar y cuándo devolver `True`.

RAISE	El servicio **avisa** que algo está mal lanzando una excepción. No muestra nada: no sabe que hay pantalla. La ventana decide cómo mostrarlo.
.LOWER()	«Ana@Correo.com» y «ana@correo.com» son el mismo usuario.
OS.URANDOM	La sal: bytes al azar. Dos personas con la misma contraseña terminan con hashes distintos.

`@staticmethod`	Igual que en `BasededatosControlador`: `_calcular_hash` no usa nada del objeto.
`600_000`	El guion bajo es solo para leer mejor el número. Tantas vueltas hacen que probar millones de contraseñas sea lentísimo.

▼ ▸ PISTA PARA AUTENTICAR()

Tres pasos: buscá al usuario en `self._usuarios` con `.get()` (si no está, devolvé `False`); calculá el hash de la contraseña que te pasaron usando **la sal guardada de ese usuario**; comparalo con el hash guardado usando `hmac.compare_digest(a, b)`, que compara sin delatar —por el tiempo que tarda— cuántos bytes coinciden.

Del lado de la ventana: atrapar el error

`ventanas/ventana_registro.py  (final de registrar)`

```
        try:
            self.servicio.registrar(datos["nombre"], datos["apellido"],
                                    datos["usuario"], datos["password"])
        except ValueError as error:
            self._mostrar_mensaje(str(error))
            return

        self._mostrar_mensaje("Cuenta creada. Ya podés volver e iniciar sesión.")
```

Atrapamos `ValueError`, **no** cualquier error: si apareciera otro tipo de problema, queremos enterarnos, no esconderlo. El login es parecido: si `autenticar()` da `True`, «Sesión iniciada como … (simulación)»; si no, un mensaje genérico.

PROBÁ · ¿DÓNDE SE CREA EL SERVICIO?

1. Hacé que cada pantalla cree el suyo en su `__init__`: `self.servicio = ServicioAutenticacion()`.
2. Registrate, volvé al login y entrá con esos datos. ¿Funciona?
3. Explicá por qué no. Pista: contá cuántos objetos `ServicioAutenticacion` hay y cuántos diccionarios.

`ventanas/ventana_principal.py  (la solución)`

```
    def __init__(self):
        super().__init__()
        servicio = ServicioAutenticacion()
        self.login = VentanaLogin(servicio)
        self.registro = VentanaRegistro(servicio)
        ...
```

Un solo servicio, creado por quien conoce a las dos pantallas, y **entregado** a cada una por su constructor. Cada ventana lo guarda: `def __init__(self, servicio):` … `self.servicio = servicio`. Las dos pantallas **tienen** el mismo servicio: composición.

PROBÁ · BUSCÁ TU CONTRASEÑA

Después de registrarte, agregá un `print(self.servicio._usuarios)` en algún método y mirá la terminal. ¿Encontrás tu contraseña? Registrá dos usuarios con **la misma** contraseña: ¿los hashes coinciden? ¿Por qué? Sacá el `print` cuando termines.

LO QUE ESTA SIMULACIÓN NO ES

- **No persiste:** al cerrar la app, los usuarios desaparecen.
- **No limita intentos:** alguien puede probar contraseñas sin parar.
- **No maneja sesiones:** «sesión iniciada» es un mensaje, no un estado de la aplicación.
- **No es un sistema seguro:** tu README tiene que decirlo con todas las letras.

PENSÁ

¿Por qué el login responde «Usuario o contraseña incorrectos» y no «Ese usuario no existe»?

El servicio vuelve a revisar el largo de la contraseña aunque la pantalla ya lo hizo. ¿Es duplicación innecesaria?

INVESTIGÁ

¿Por qué un SHA-256 solo, sin sal ni vueltas, no alcanza para contraseñas? ¿Qué son **bcrypt** y **Argon2**? ¿Qué es un ataque de diccionario?

COMPROBÁ

- [ ] Registrarse y después iniciar sesión con esos datos funciona.
- [ ] Con una contraseña incorrecta, el mensaje no dice cuál de los dos datos falló.
- [ ] Un usuario repetido muestra el error del servicio y la app sigue andando.
- [ ] En `_usuarios` no aparece ninguna contraseña legible.
- [ ] `autenticacion.py` no importa nada de PyQt6.

09

Mejorar la experiencia visual

Recién ahora, con todo funcionando: que además se entienda y se vea bien

NIVEL 2 · AL MENOS TRES MEJORAS

EL PROBLEMA

Funciona. Pero el error está en el mismo gris que todo lo demás, el botón principal no se distingue del secundario y, en pantalla completa, los campos se estiran de borde a borde.

LA IDEA · QT STYLE SHEETS

Qt tiene su propio lenguaje de estilos, **QSS**. Se escribe parecido a CSS —y eso ayuda— pero **no es CSS**:

	CSS (WEB)	QSS (QT)
Qué estiliza	Elementos HTML	Widgets de Qt
Selector de tipo	`button`, `input`	`QPushButton`, `QLineEdit`
Selector por nombre	`#id`	`#objectName`
Estados	`:hover`, `:focus`, `:disabled`	Los mismos, más `:pressed` y `:checked`
Por atributo	`[type="password"]`	`[propiedad="valor"]`, con propiedades de Qt
Ubicar y alinear	flexbox, grid	**No.** Eso lo hacen los layouts.
Animaciones	`transition`	**No existen.**
Sombras	`box-shadow`	**No existe.** (Hay un efecto aparte, desde Python.)
Degradés	`linear-gradient()`	`qlineargradient()`, con otra sintaxis

Si escribís una propiedad de CSS que Qt no conoce, **se ignora sin avisar**. La referencia es la página «Qt Style Sheets Reference» de la documentación de Qt.

¿Dónde se escriben?

EN PYTHON

`widget.setStyleSheet("…")`. Mezcla apariencia y comportamiento en el mismo archivo. Evitalo.

EN DESIGNER

La propiedad `styleSheet` del formulario. Sirve para probar, pero queda copiado en cada `.ui`: dos pantallas, dos copias.

EN UN ARCHIVO · ELEGIMOS ESTE

`ventanas/estilos.qss`, cargado **una vez** para toda la app. Un cambio, un lugar.

`main.py  (lo que se agrega)`

```python
from pathlib import Path

RUTA_ESTILOS = Path(__file__).parent / "ventanas" / "estilos.qss"


def cargar_estilos(app):
    try:
        app.setStyleSheet(RUTA_ESTILOS.read_text(encoding="utf-8"))
    except FileNotFoundError:
        print(f"Aviso: no encontré {RUTA_ESTILOS}. Sigo con el estilo por defecto.")

# y en main(), después de crear app:   cargar_estilos(app)
```

Manejo de errores con criterio: si falta el archivo de estilos, la app **se ve peor pero funciona**, y avisa por qué. No hay motivo para que un estilo perdido impida iniciar sesión.

LA DEMOSTRACIÓN · DE A UN BLOQUE

```
/* el botón principal y sus estados */
QPushButton#btn_login {
    background-color: #1548c8;
    color: white;
    border: none;
    border-radius: 8px;
    padding: 11px 16px;
    font-weight: 600;
}
QPushButton#btn_login:hover    { background-color: #1240ad; }
QPushButton#btn_login:pressed  { background-color: #0e3590; }
QPushButton#btn_login:disabled { background-color: #9aa9bd; }
```

```
/* campos: borde suave, y marcado al escribir */
QLineEdit {
    background-color: white;
    color: #111a26;
    border: 1px solid #ccd4e0;
    border-radius: 8px;
    padding: 8px 10px;
}
QLineEdit:focus { border: 2px solid #1548c8; }

/* la tarjeta y el fondo */
QFrame#frame_tarjeta {
    background-color: #fbfcfd;
    border: 1px solid #ccd4e0;
    border-radius: 16px;
}
QStackedWidget { background-color: #e7eaef; }
```

DOS TRAMPAS QUE VAS A PISAR

- `QLabel` **hereda de** `QFrame`**.** Si escribís `QFrame { border: 1px solid gray; }`, le ponés borde a **todas las etiquetas**. Por eso usamos `QFrame#frame_tarjeta`. La herencia, otra vez, ahora en los estilos.
- **Modo oscuro.** Si pintás un fondo claro y no definís el color del texto, en una compu con modo oscuro podés terminar con texto claro sobre fondo claro. Si definís un fondo, definí también `color` para `QLabel` y `QCheckBox`.

Mensajes de error y de éxito, sin mezclar responsabilidades

```
    def _mostrar_mensaje(self, texto, tipo="info"):
        self.label_mensaje.setText(texto)
        self.label_mensaje.setProperty("tipo", tipo)
        self.label_mensaje.style().unpolish(self.label_mensaje)
        self.label_mensaje.style().polish(self.label_mensaje)
```

```
QLabel#label_mensaje[tipo="error"] {
    color: #b7361b;
    font-weight: 600;
}
QLabel#label_mensaje[tipo="exito"] {
    color: #0f7d55;
    font-weight: 600;
}
```

Python dice **qué tipo** de mensaje es; el QSS decide **cómo se ve**. Si mañana los errores van en naranja, no se toca Python. `unpolish` y `polish` le piden a Qt que vuelva a mirar los estilos: un cambio de propiedad hecho después de dibujar no se aplica solo. Y como el parámetro tiene valor por defecto, las llamadas que ya tenías siguen funcionando.

Elegí al menos tres mejoras

COLORES Y TIPOGRAFÍA

Una paleta corta (fondo, tarjeta, texto, acento, error, éxito) y una jerarquía clara: título, etiquetas, ayuda.

ESTADOS DE BOTONES

`:hover`, `:pressed`, `:disabled`. Bonus: deshabilitar el botón mientras falten datos.

MOSTRAR CONTRASEÑA

Si ya está con la casilla, llevalo más lejos: un ícono de ojo adentro del campo (investigá `QLineEdit.addAction`).

MENSAJES

Error y éxito distinguibles **por el texto** además del color: no todo el mundo distingue rojo de verde.

ESPACIADO Y ALINEACIÓN

En Designer, las propiedades `layoutSpacing` y los márgenes de cada layout. Aire entre grupos, no entre todo.

CAMPOS OBLIGATORIOS

Un `*` en la etiqueta y, al fallar, el borde del campo en rojo con una propiedad (`[invalido="true"]`).

TRANSICIÓN CLARA

Título de ventana distinto por pantalla, foco en el primer campo y mensajes limpios al cambiar.

REGISTRO DE DECISIONES DE DISEÑO

Una fila por mejora. Sacá una captura antes y otra después, y guardalas en `capturas/`: van a la entrega.

MEJORA	POR QUÉ LA ELEGISTE	DÓNDE (QSS / DESIGNER / PYTHON)	CAPTURAS
1			

MEJORA	POR QUÉ LA ELEGISTE	DÓNDE (QSS / DESIGNER / PYTHON)	CAPTURAS
2			
3			

PENSÁ

- ¿Por qué el estilo va en un archivo aparte y no adentro de cada `.ui`?
- Un efecto visual llamativo que hace más difícil leer un mensaje de error, ¿es una mejora?

INVESTIGÁ

En la «Qt Style Sheets Reference», encontrá: ¿qué propiedades admite `QLineEdit`? ¿Existe `text-transform`? ¿Qué son los subcontroles como `::indicator` de un `QCheckBox`, y qué podrías hacer con ellos?

COMPROBÁ

- [ ] Implementé al menos tres mejoras y las registré, con capturas antes y después.
- [ ] Los siete casos de la etapa 7 siguen dando el mensaje esperado.
- [ ] Error y éxito se distinguen por el texto, no solo por el color.
- [ ] Si renombro `estilos.qss`, la app abre igual y avisa en la terminal.

10

Organización del proyecto

Cada archivo existe porque resuelve algo

NIVEL 2

EL PROBLEMA

Si todo esto viviera en `main.py` serían más de trescientas líneas, con las dos pantallas mezcladas, y para cambiar la regla de la contraseña habría que buscarla entre botones y colores.

LA ESTRUCTURA

```
login-registro/
├── main.py                    arranca la app y carga los estilos
├── ventanas/
│   ├── login.ui               diseño del login (Designer)
│   ├── registro.ui            diseño del registro (Designer)
│   ├── estilos.qss            apariencia de toda la app
│   ├── ventana_login.py       VentanaLogin: qué hace el login
│   ├── ventana_registro.py    VentanaRegistro: qué hace el registro
│   └── ventana_principal.py   VentanaPrincipal: qué pantalla se ve
├── controladores/
│   └── autenticacion.py       ServicioAutenticacion (simulado)
├── capturas/                  imágenes para el README
├── .gitignore
├── requirements.txt
└── README.md
```

Cada diseño vive **al lado de la clase que lo controla**: `login.ui` ↔ `ventana_login.py`. Por eso la ruta al `.ui` es tan corta. Y no hay `__init__.py`: Python 3 reconoce estas carpetas como paquetes sin él, y no agregamos archivos vacíos que no resuelven nada.

ARCHIVO	SABE	NO SABE
`main.py`	Arrancar la app y aplicar los estilos.	Que existen botones o usuarios.
`*.ui`	Qué widgets hay, dónde y cómo se llaman.	Qué pasa al tocarlos.
`estilos.qss`	Cómo se ve todo.	Qué hace nada.
`ventana_login.py` `ventana_registro.py`	Qué hacer con los eventos de su pantalla y qué mensaje mostrar.	Cómo se guardan los usuarios. Que existe la otra pantalla.
`ventana_principal.py`	Qué pantallas hay, cómo se pasa de una a otra, y quién comparte el servicio.	Qué valida cada pantalla.
`autenticacion.py`	Registrar y verificar usuarios, y las reglas del sistema.	Que existe PyQt.
`requirements.txt`	Qué instalar para **ejecutar**.	Qué usaste para diseñar.
`README.md`	Cómo instalar, ejecutar, qué hace y qué **no** hace.	—

Las dependencias van para un solo lado



ERROR CLÁSICO DE ESTA ETAPA

Ejecutar `python ventanas/ventana_registro.py` da `ModuleNotFoundError: No module named 'controladores'`. Python busca los módulos a partir de la carpeta del archivo que ejecutaste, y desde `ventanas/` no se ve `controladores/`. **Siempre se ejecuta `main.py`, parado en la carpeta del proyecto.**

PROBÁ · QUE EL SERVICIO NO CONOCE PYQT

Mac: `grep -n PyQt6 controladores/autenticacion.py` · Windows: `findstr PyQt6 controladores\autenticacion.py`. Si no imprime nada, está bien. Si imprime algo, la lógica se mezcló con la interfaz.

LO QUE NO HACE FALTA

Carpetas `utils/`, `models/`, `interfaces/` o `base/` vacías «por las dudas», clases abstractas sin un segundo uso, capas que solo pasan datos de una a otra. La organización **resuelve problemas que tenés**, no los que podrías tener.

PENSÁ

Si mañana hicieras una versión de consola del mismo login, ¿qué archivos reutilizarías tal cual? ¿Cuáles tirarías?

COMPROBÁ

- [ ] Mi proyecto tiene esta estructura, o el README explica cada diferencia.

- [ ] `main.py` tiene menos de 30 líneas y no menciona ningún widget.
- [ ] El servicio no importa PyQt6 y no hay archivos vacíos ni carpetas sin uso.

11 Buenas prácticas y seguridad

Una revisión de código: primero la de otro, después la tuya

NIVEL 1

Las buenas prácticas ya aparecieron en cada etapa. Ahora las vas a **reconocer** en código ajeno. Este programa «funciona»: abre una ventana, deja entrar y registra usuarios. Leelo como si tuvieras que aprobarlo antes de que llegue a producción.

`app_de_un_compañero.py  (no lo copies)`

```
from PyQt6 import uic
from PyQt6.QtWidgets import QApplication
import sys

usuarios = {"admin": "admin123"}
app = QApplication(sys.argv)
v = uic.loadUi("login.ui")

def f():
    global usuarios
    u = v.lineEdit.text()
    p = v.lineEdit_2.text()
    if u in usuarios and usuarios[u] == p:
        v.label.setText("Bienvenido!!")
        v.label.setStyleSheet("color: green")
    else:
        v.label.setText("Error")
        v.label.setStyleSheet("color: red")

def g():
    u = v.lineEdit_3.text()
    p = v.lineEdit_4.text()
    edad = eval(v.lineEdit_5.text())
    try:
        usuarios[u] = p
        v.label_2.setText("ok")
    except:
        pass

v.pushButton.clicked.connect(f)
v.pushButton_2.clicked.connect(g)
v.show()
app.exec()
```

PROBÁ · ENCONTRÁ AL MENOS DIEZ PROBLEMAS

Para cada uno: qué está mal, por qué importa y cómo lo resolviste vos en tu proyecto.

▼ ▸ COMPARAR CON LA LISTA (DESPUÉS DE ESCRIBIR LA TUYA)

PROBLEMA	POR QUÉ IMPORTA	EN ESTE TP
Nombres `f`, `g`, `v`, `lineEdit_2`	Nadie sabe qué hace cada cosa sin ejecutarla.	`iniciar_sesion`, `input_password`.
Todo en un archivo, sin clases	Interfaz, reglas y datos mezclados: cualquier cambio toca todo.	Una clase por pantalla y un servicio.
Variables globales (`usuarios`, `v`, `global`)	Cualquier función puede cambiarlas; los errores aparecen lejos de su causa.	Atributos: `self.servicio`, `self._usuarios`.
`"admin123"` escrito en el código	Una contraseña real en el código **termina en GitHub**, a la vista.	Ningún usuario precargado; secretos en `.env`, fuera de Git.
Contraseñas guardadas como texto	Quien vea los datos ve todas las contraseñas.	Hash con sal (etapa 8).
`eval()` sobre lo que escribe el usuario	Ejecuta **cualquier** código: escribí `__import__("os").getcwd()` y lo corre. Con otra línea podría borrar archivos.	Nunca `eval()`. Para un número: `int()` dentro de `try` / `except ValueError`.
`except: pass`	Se traga cualquier error, incluso los que no imaginaste. El programa falla en silencio.	`except ValueError as error:` y se muestra el mensaje.
Sin validación	Registra usuarios vacíos y no confirma la contraseña.	Etapa 7.
«Bienvenido!!» / «Error»	Uno promete más de lo que hay; el otro no dice qué hacer.	Mensajes honestos y accionables.
Estilos con `setStyleSheet` repetidos	Apariencia mezclada con lógica y duplicada.	`estilos.qss` y la propiedad `tipo`.

PROBLEMA	POR QUÉ IMPORTA	EN ESTE TP
Ruta relativa `"login.ui"`	Falla según desde dónde se ejecute.	`Path(__file__).parent`.
Sin `sys.exit()` **ni** `if __name__ == "__main__"`	Importar el archivo abre la ventana.	`main()` protegido.

SECRETOS Y GIT

Si una contraseña llegó a un commit, borrarla en el commit siguiente **no alcanza**: queda en el historial, y el historial se sube entero. Lo correcto es **cambiar esa contraseña**. Por eso lo sensible va en `.env` desde el principio, y el `.env` en el `.gitignore`, como en la guía de MySQL.

Auditá tu propio código

- [ ] Cada nombre dice qué es o qué hace. No hay `f`, `aux`, `lineEdit_2` ni `pushButton`.
- [ ] Cada método hace una sola cosa y entra en la pantalla sin hacer scroll.
- [ ] No hay bloques copiados y pegados. (Si quedó alguno a propósito, sé explicar por qué.)
- [ ] Los `.ui` no tienen comportamiento y el Python no tiene posiciones ni colores.
- [ ] `main.py` solo arranca la aplicación.
- [ ] No hay variables globales ni la palabra `global`.
- [ ] Cada `except` nombra el error que espera. No hay `except: pass`.
- [ ] No hay `eval()` ni `exec()`.
- [ ] Ninguna contraseña se guarda ni se imprime como texto. No quedaron `print` de prueba.
- [ ] No hay contraseñas reales, tokens ni datos personales en el código ni en el historial de Git.
- [ ] El README dice que la autenticación es simulada y por qué no es segura.

12 Desafíos progresivos

Tres niveles. El primero es obligatorio; el tercero, una invitación a investigar

NIVEL 1 · OBLIGATORIO

- [ ] Login y registro **diseñados en Qt Designer**, con layouts y los nombres de la convención.
- [ ] Las dos pantallas se **cargan desde** `.ui` con `uic.loadUi()` y una ruta construida con `Path`.
- [ ] Una **clase propia** por pantalla, con `super().__init__()` y `self` bien usados.
- [ ] Los botones **conectados** a métodos de esas clases.

- [ ] Los campos **validados** según la etapa 7.
- [ ] **Mensajes** claros y honestos: ninguno promete una autenticación que no existe.

NIVEL 2 · INTEGRACIÓN

- [ ] **Alternar** entre las dos pantallas en una sola ventana (`VentanaPrincipal`).
- [ ] **Mostrar u ocultar** la contraseña (en el registro, las dos).
- [ ] Un formulario inválido **nunca se procesa**: ni registra ni autentica.
- [ ] Registro y login **simulados** con un servicio compartido y contraseñas con hash.
- [ ] Al menos **tres mejoras visuales** registradas (etapa 9).
- [ ] Código **organizado en módulos** según la etapa 10.

NIVEL 3 · ¿Y SI...? (OPCIONAL)

No están resueltos, a propósito. Elegí uno, investigalo y documentá en el README qué hiciste, qué encontraste y qué no pudiste resolver. **Un desafío a medias, bien explicado, vale más que tres copiados.**

¿Y SI... AGREGAMOS «RECORDAR MI USUARIO»?

Una casilla en el login. Al volver a abrir la app, el usuario ya aparece escrito.

▼ ▸ PISTAS

Investigá `QSettings` : guarda valores chicos entre una ejecución y otra, en el lugar que corresponde a cada sistema operativo. Mirá `setValue()` y `value()` . ¿Dónde los guarda en Windows y dónde en Mac?

Se recuerda **el usuario**. La contraseña, jamás.

¿Y SI... MOSTRAMOS LA FORTALEZA DE LA CONTRASEÑA?

Mientras se escribe en el registro, un indicador dice si es débil, aceptable o fuerte.

▼ ▸ PISTAS

¿Qué señal de `QLineEdit` avisa cada vez que cambia el texto? Conectala a un método que calcule un puntaje y lo muestre en un `QLabel` o un `QProgressBar` .

Para el puntaje: largo, si tiene números (`any(c.isdigit() for c in texto)`), mayúsculas, símbolos. ¿Dónde va ese cálculo: en la ventana o en el servicio? ¿Por qué?

Pensá: un medidor orienta, no garantiza nada. ¿Es más fuerte `Ab1!` o `caballo correcto batería grapa` ?

¿Y SI... AGREGAMOS UNA PANTALLA PARA RECUPERAR LA CONTRASEÑA?

Un enlace «¿Olvidaste tu contraseña?» que lleva a una tercera pantalla.

▼ ▸ PISTAS

Es un `.ui` más, una clase más y un `addWidget` más en `VentanaPrincipal`. Si tuviste que tocar las otras dos pantallas para agregarla, algo de la organización se puede mejorar.

Pensá qué haría falta de verdad: un correo, un código de un solo uso, que venza. Acá **no** simules que se envía un mail. Y fijate el mensaje: «Si el usuario existe, te enviaremos instrucciones» no revela quién está registrado. ¿Por qué importa?

¿Y SI... NECESITAMOS REGISTRAR LOS DATOS EN MYSQL?

Que los usuarios sigan ahí al cerrar la app.

▼ ▸ PISTAS

Volvé a las secciones 8, 9 y 11 de la guía de Python + MySQL. Pensá una tabla `usuarios`: ¿qué columnas necesita? ¿Cuál tiene que ser única? ¿Dónde van la sal y el hash? (Investigá `VARBINARY`, o convertí los bytes con `.hex()`.)

La pregunta que importa: **¿cuántos archivos tuviste que tocar?** Si las ventanas cambiaron, la separación no estaba bien hecha. Solo debería cambiar el interior de `ServicioAutenticacion`.

Todas las consultas con `%s`. Y los datos de conexión, en el `.env`.

¿Y SI... VARIAS PANTALLAS NECESITAN LA MISMA LÓGICA DE AUTENTICACIÓN?

Después del login aparece una pantalla «Mi perfil» que necesita el servicio y saber **quién** inició sesión.

▼ ▸ PISTAS

¿Quién crea el servicio? ¿Quién se lo pasa a la pantalla nueva? ¿Qué pasaría si la pantalla nueva creara el suyo? (Ya lo viviste en la etapa 8.)

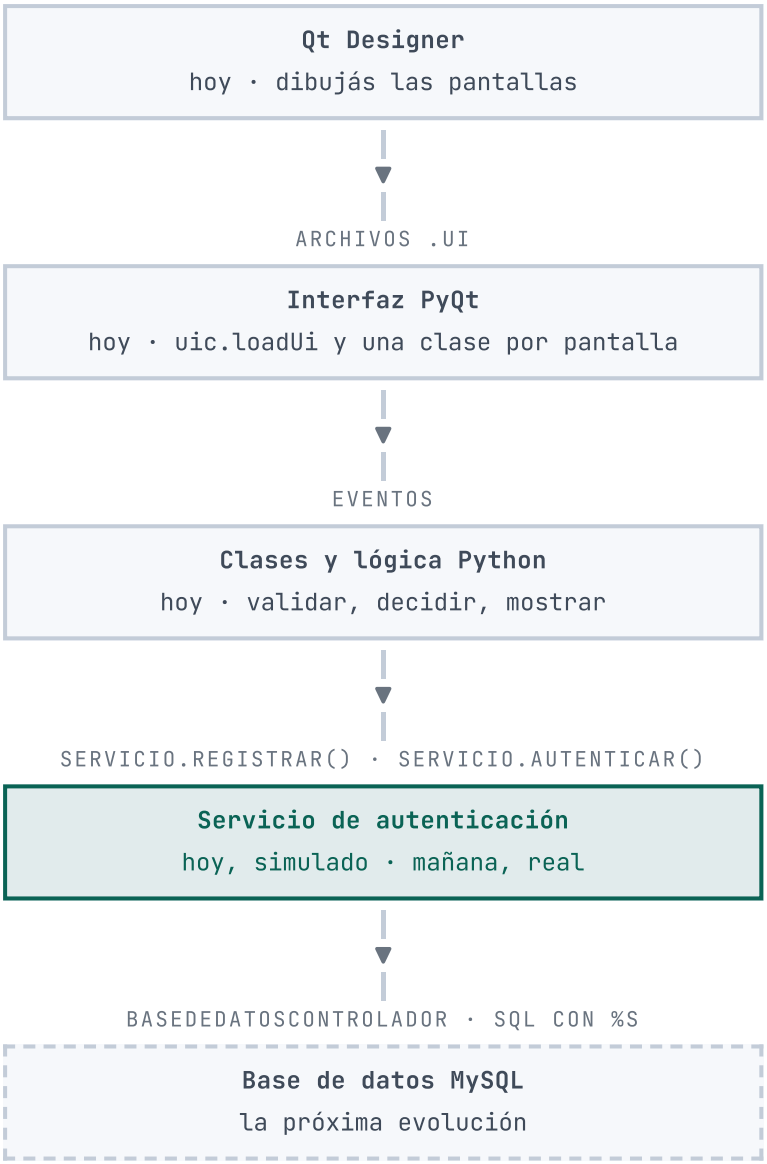
¿Cómo se entera `VentanaPrincipal` de que el login salió bien, sin meterse en los botones del login? Investigá `pyqtSignal`: una pantalla puede **emitir su propia señal** («sesión iniciada», con el usuario) y la ventana principal conectarse a ella. Con eso también podés resolver el «Pensá» de la etapa 6.

Y si `_mostrar_mensaje` está copiado en tres pantallas: ¿una clase padre `Pantalla(QWidget)` que cargue el `.ui` y sepa mostrar mensajes? Herencia que nace de duplicación real, no «por las dudas».

13

Conexión con el futuro

Este TP prepara el terreno; no es todavía un sistema de usuarios



Cuando llegue MySQL, ¿qué cambia?

PIEZA	HOY	CON MYSQL
`login.ui`, `registro.ui`	El diseño.	**Igual.**
`VentanaLogin`, `VentanaRegistro`	Llaman a `registrar()` y `autenticar()`.	**Igual.**
`VentanaPrincipal`	Crea un servicio y lo comparte.	**Igual** (o casi).
`ServicioAutenticacion`	Un diccionario en memoria.	**Cambia por dentro:** consultas a través de `BasededatosControlador`.
Las contraseñas	Hash con sal, en memoria.	Hash con sal, en columnas. **Nunca** la contraseña.

LA PRUEBA DE QUE LO HICISTE BIEN

Si cuando llegue la base de datos tenés que tocar las ventanas, la separación no estaba bien hecha. Es la misma frase con la que cerraba la guía de MySQL, ahora vista desde el otro lado: **la interfaz no tiene por qué saber cómo se guardan los usuarios. Solo pide.**

LO QUE TODAVÍA FALTA PARA UN SISTEMA REAL (Y ESTÁ BIEN QUE FALTE)

- Persistencia: que los usuarios sobrevivan al programa.
- Límite de intentos y bloqueo temporal ante muchos errores seguidos.
- Sesiones: recordar quién entró, por cuánto tiempo, y poder cerrar sesión.
- Verificar el correo y recuperar la contraseña con enlaces que vencen.
- Roles y permisos: no todos pueden hacer lo mismo.
- Si la app habla con un servidor: conexión cifrada y validación del lado del servidor.

14

Entrega y GitHub

Que cualquiera pueda clonarlo, instalarlo y abrirlo sin preguntarte nada

Cada estudiante o pareja (máximo dos personas) entrega **el link a un repositorio de GitHub** con:

1 · DISEÑO

`ventanas/login.ui`, `ventanas/registro.ui` y, si lo usaste, `estilos.qss`.

2 · CÓDIGO

`main.py`, las clases de `ventanas/` y `controladores/autenticacion.py`.

3 · REQUIREMENTS.TXT

Lo necesario para ejecutar. Nada más.

4 · README.MD

Instalación, ejecución, funcionalidades, limitaciones y decisiones.

5 · CAPTURAS

En `capturas/`: las dos pantallas, un error, un éxito y el antes/después de tus mejoras.

6 · EVIDENCIA

La checklist de funcionalidades y, si hiciste alguno, el desafío del nivel 3 explicado.

requirements.txt

`requirements.txt`

```
PyQt6==6.11.0
```

Poné la versión que tenés instalada: la ves con `python -m pip show PyQt6`. ¿Por qué no `pip freeze > requirements.txt`? Porque también anota `PySide6-Essentials` y `shiboken6`, que son de Designer: quien ejecuta tu app no los necesita.

README.md · una plantilla para empezar

```
# Login y registro con PyQt6

Aplicación de escritorio con dos pantallas diseñadas en Qt Designer y controladas
con clases de Python. Trabajo práctico de Programación Orientada a Objetos.

> **Importante:** la autenticación es una SIMULACIÓN. Los usuarios viven en
> memoria y se pierden al cerrar la aplicación. No es un sistema seguro ni
> apto para datos reales.

## Capturas
![Login](capturas/login.png)
![Registro](capturas/registro.png)

## Requisitos
- Python 3.10 o más nuevo

## Instalación
Windows (PowerShell):
    py -m venv .venv
    .venv\Scripts\Activate.ps1
    python -m pip install -r requirements.txt

macOS:
    python3 -m venv .venv
    source .venv/bin/activate
    python -m pip install -r requirements.txt

## Ejecución
    python main.py

## Funcionalidades
- [x] ...
- [ ] ...

## Desafíos realizados
## Decisiones de diseño
## Limitaciones conocidas
## Autores
```

Git, paso a paso

```
git init                        # solo la primera vez
git status                      # ¿qué cambió? ¿aparece .venv? (no debería)
git add main.py ventanas controladores capturas
git add requirements.txt README.md .gitignore
git status                      # revisá, archivo por archivo, lo que va a entrar
git commit -m "feat: agrega login y registro con PyQt y Qt Designer"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/login-registro-pyqt.git
git push -u origin main
```

Creá antes el repositorio vacío en GitHub (sin README, para que no choque con el tuyo). Mejor que un solo commit gigante al final: **un commit por etapa**, con el formato del curso, `tipo: qué hace`, en presente: `feat: agrega la pantalla de registro` · `fix: corrige la ruta del .ui` · `docs: completa el README`.

ANTES DEL PUSH

- [ ] `git status` no muestra `.venv/`, `__pycache__/` ni `.env`.
- [ ] No hay contraseñas reales, tokens ni datos personales en ningún archivo ni captura.
- [ ] El README dice que la autenticación es simulada.
- [ ] **Prueba de cero:** cloné mi repo en otra carpeta, creé el entorno, instalé `requirements.txt` y `python main.py` abrió la app.

SI TRABAJARON EN PAREJA

Las dos personas tienen que poder explicar **todo** el proyecto, no solo la parte que escribieron. En la defensa se le pregunta a cualquiera de las dos, sobre cualquier archivo. Y el historial de Git muestra quién hizo cada commit.

15 Autoevaluación y criterios

Marcá solo lo que podrías hacer sin mirar la guía

- [ ] Puedo explicar la diferencia entre instalar PyQt6 y tener Qt Designer.
- [ ] Sé crear y activar un entorno virtual en mi sistema operativo.
- [ ] Sé diseñar una pantalla en Designer con layouts, sin posiciones a mano.
- [ ] Puedo explicar la diferencia entre el texto visible de un widget y su `objectName`.
- [ ] Sé cargar un `.ui` con `uic.loadUi()` y una ruta que no depende de dónde ejecuto.
- [ ] Puedo explicar qué hace el bucle de eventos.
- [ ] Puedo justificar por qué mi pantalla hereda de `QWidget` y qué pasa si saco `super().__init__()`.
- [ ] Sé conectar una señal a un método y por qué va sin paréntesis.
- [ ] Sé validar un formulario y cortar el proceso cuando algo falla.
- [ ] Puedo explicar por qué validar el formulario no es autenticar.

- [ ] Puedo explicar qué es un hash con sal y por qué no se guarda la contraseña.
- [ ] Sé escribir estilos en QSS y en qué se diferencia de CSS.
- [ ] Puedo decir qué sabe y qué no sabe cada archivo de mi proyecto.
- [ ] Sé leer un traceback y encontrar la línea de mi código que lo provocó.
- [ ] Sé preparar un commit revisando antes qué entra y qué no.

REFLEXIÓN FINAL

¿Qué fue lo más difícil de conectar entre el diseño y el código? ¿Cómo lo resolviste?

Si empezaras de nuevo, ¿qué decidirías distinto?

Cómo se evalúa · criterios para el docente

Como en Arena de Combate, se evalúa principalmente **el razonamiento y la comprensión**, no la cantidad de funcionalidades. Una aplicación chica que se entiende a fondo vale más que una enorme que no se sabe explicar. **Para aprobar:** nivel 1 completo y defendido. **Para una nota alta:** nivel 2 completo y un desafío del nivel 3 bien explicado.

QUÉ SE MIRA	LO ESPERADO	SEÑAL DE ALERTA
Diseño en Designer	Layouts en todos los contenedores; nombres de la convención; tarjeta centrada; orden de Tab pensado.	Widgets ubicados a mano; `lineEdit_2`; texto cortado al cambiar el tamaño.
Diseño y comportamiento separados	`.ui` cargados con `loadUi`; nada de lógica en el diseño ni de estilos en Python.	Un `.py` generado con `pyuic6` editado a mano; `setStyleSheet` desparramado.
POO	Una clase por pantalla; herencia que se puede justificar; `super()` y `loadUi` en orden; el servicio compartido por composición.	Variables globales; todo en `main.py`; herencia que no sabe explicar.
Eventos	Las cinco conexiones funcionan; usa bien una señal que trae un dato (`toggled`).	Conexiones que se disparan al abrir la ventana.
Validación y mensajes	Los casos de la etapa 7 pasan; cláusulas de guarda; mensajes claros, accionables y honestos.	«Bienvenido» sin verificar; errores que no dicen qué hacer.

QUÉ SE MIRA	LO ESPERADO	SEÑAL DE ALERTA
Simulación y seguridad	Hash con sal; mensaje genérico ante un login fallido; el README declara la simulación; nada sensible en Git.	Contraseñas en texto plano; contraseñas reales en el código; `.venv` o `.env` subidos; `eval()`.
Organización	Estructura de la etapa 10; el servicio no importa PyQt; métodos cortos; sin duplicación evitable.	Carpetas vacías; un método que hace todo.
Experiencia visual	Al menos tres mejoras justificadas; la claridad está antes que el efecto.	Mejoras sin registro ni capturas; errores que solo se distinguen por el color.
Entrega	README completo; capturas; `requirements.txt` mínimo; se instala de cero sin ayuda.	«En mi compu anda».
Explicar las decisiones	**Lo que más pesa.** Puede contar por qué diseñó así, qué otra opción había y qué tocaría para agregar MySQL.	Código que funciona y que no sabe explicar.

Antes de cerrar la guía

1. Instalar **PyQt6** te da una librería; instalar **Designer** te da una herramienta. El `.ui` es el puente entre las dos.
2. El `.ui` **describe**; Python **decide**. Lo único que comparten es el `objectName`.
3. `uic.loadUi(ruta, self)` convierte el diseño en atributos de tu objeto. Por eso la pantalla hereda de `QWidget`.
4. **Señal → slot**: `widget.señal.connect(self.metodo)`, sin paréntesis.
5. Leer, **validar**, cortar o seguir. Validar ayuda a la persona; no reemplaza al servidor ni a la autenticación.
6. Una contraseña **nunca** se guarda: se guarda su hash con sal. Y una simulación se presenta como simulación.
7. Cada archivo sabe una cosa y **no sabe** las demás. Así, cuando llegue MySQL, cambia un solo lugar.

Diseñamos visualmente, programamos el comportamiento y conectamos ambas partes con buenas prácticas.

Si al terminar podés abrir Designer, cambiar una pantalla y saber exactamente qué parte del código se ve afectada —y cuál no—, el trabajo práctico cumplió su objetivo.

Trabajo práctico · Login y registro · PyQt6 + Qt Designer · Técnico Universitario en Programación

Continúa la guía de exploración de PyQt y la guía de Python + MySQL